

hyptcn3 — všetko o projekte

Hypervisor modul pre real-time RAM introspekciu s využitím Temporal Convolutional Networks

Podklad pre NotebookLM

Bc. Erik Horváth · UKF Nitra, FPVaI

Stav repozitára k 19. septembru 2026

Contents

Ako to celé funguje: hyptcn3 od nuly po obhajobu	1
1. O čom je táto práca a prečo	1
2. Virtualizácia, hypervízor a kde v tom sedí náš nástroj	3
3. Zberač: ako sa dostaneme k pamäti bežiacého stroja	7
4. Ostatné časti zberača, ktoré sa dajú ukázať	11
5. Semantic gap: ako z bajtov spravíme procesy	14
6. Príznaky a model: od snímky k číslam a k TCN	21
7. Ako sa to preloží a spustí	26
8. Čo máme namerané	28
9. Obmedzenia L1 až L17	34
10. HONESTY.md: pravidlá, strojová kontrola a prečo sa projekt začal nanovo	36
11. Stav voči zadaniu a obhajoba	38
12. Slovníček	40

Ako to celé funguje: hyptcn3 od nuly po obhajobu

Tento dokument je určený jednému čitateľovi: autorovi práce pred obhajobou. Začína sa od nuly a každé prirovnanie je hneď preložené na skutočný mechanizmus a skutočné meno — funkciu, štruktúru, súbor v repozitári. Prirovnanie samo o sebe pred komisiou neobstojí; obstojí veta „je to takto, volá sa to takto a je to v tomto súbore“.

Dve pravidlá, ktoré platia pre celý text. Každé číslo je buď z uloženého artefaktu v `data/results/`, alebo z behu, ktorý je pri ňom pomenovaný aj s príkazom. Číslo bez jedného z týchto dvoch v texte nie je. A druhé: tam, kde sa niečo nezmeralo, je to napísané slovom „nezmerané“ alebo značkou `UNVERIFIED`, nie opatrnou formuláciou.

1. O čom je táto práca a prečo

1.1 Čo doslovne žiada zadanie

Názov práce znie: „*Návrh a implementácia hypervisor modulu pre real-time RAM introspekciu s využitím Temporal Convolutional Networks*“ (zadanie-zp_105826.pdf). Päť pojmov, jeden po druhom.

Virtuálny stroj je celý počítač vyrobený softvérom. Má vlastné jadro operačného systému, vlastné procesy, vlastnú sieť. Beží pritom ako obyčajný program na inom, fyzickom počítači. Fyzickému stroju sa hovorí **hostiteľ** (host), virtuálnemu **host** (guest).

Hypervízor je program, ktorý virtuálne stroje prevádzkuje: prideliť im pamäť a procesor a rozhoduje, kto kedy beží. Je to správca budovy, ktorý má kľúče od všetkých bytov — nájomník o tom nevie a nevie mu to zakázať. Po odložení prirovnania: v tejto práci je hypervízorom **KVM**, ktorý je na použítom hostiteľovi zavedený ako modul jadra Linuxu (`lsmod | grep '^kvm'` vypíše `kvm` a `kvm_intel`). KVM drží pamäť hosta ako zoznam blokov; jeden blok sa volá **memslot** a leží v tabuľke `memslotov` v štruktúre `struct kvm`. `Memslot` hovorí, ktorý rozsah

fyzických adres host'a je na ktorej adrese v procese QEMU. Meraná doména `hyptcn-guest` mala 10 memslotov a 2,02 GiB RAM (`data/results/probe_20260919T142627Z.json`).

RAM introspekcia znamená čítať operačnú pamäť bežiaceho virtuálneho stroja zvonka, z hostiteľa, bez pomoci čohokoľvek vnútri host'a.

Real-time je v zadaní slovo bez čísla. Pravidlá tohto repozitára (`HONESTY.md`, P7) ho zakazujú používať samostatne, takže sa v texte nahrádza konkrétnou periódou zberu a nameranou latenciou. Prakticky ide o periodický zber s meraným časom cyklu.

Temporal Convolutional Network (TCN) je druh neurónovej siete určený pre postupnosti — pre dáta, ktoré idú za sebou v čase.

Kľúčová veta anotácie znie: „*Cieľom diplomovej práce je navrhnuť a implementovať hypervisor modul postavený na open-source hypervízore (QEMU/KVM, Xen alebo BareFlank) schopný vykonávať real-time introspekciu RAM virtuálnych strojov a generovať per-bin feature vektory pripravené na analýzu pomocou TCN pre detekciu pokročilých kybernetických hrozieb.*“

Po lopate: napíš program, ktorý beží na hostiteľovi, pravidelne odfoť pamäť virtuálneho stroja, z každej fotky spočíta krátky rad čísel a tie rady predloží neurónovej sieti, ktorá má povedať, či sa vnútri deje niečo zlé.

1.2 Prečo sa vôbec niekto pozerá dovnútra zvonku

APT (Advanced Persistent Threat) je útočník, ktorý sa do systému dostane a potom v ňom ticho zostane — mesiace, nie minúty. **Rootkit na úrovni jadra** je kód, ktorý sa zabuduje priamo do jadra operačného systému a odtiaľ prepisuje odpovede, ktoré jadro dáva ostatným programom. **Bezúborový malvér** (fileless malware) nič nezapíše na disk; žije iba v operačnej pamäti, takže po reštarte po ňom nezostane súbor, ktorý by sa dal preskenovať. Všetky tri sú v anotácii zadania menované priamo.

Antivírus **vnútri** stroja je proti nim v nevýhodnej pozícii z principiálneho dôvodu. Antivírus je program, ktorý beží v tom istom operačnom systéme a pýta sa toho istého jadra: aké procesy bežia, aké súbory tu sú, aké spojenia sú otvorené. Keď útočník ovládne jadro, ovládne aj odpovede. Antivírus sa spýta a jadro klame. Navyše ho útočník s právami jadra môže rovno vypnúť, lebo je to len ďalší proces v tabuľke procesov.

Virtual Machine Introspection (VMI) presúva pozorovateľa o poschodie nižšie — mimo dosah host'a. Zadanie to formuluje takto: VMI „*umožňuje neinvazívne monitorovanie stavu pamäte, procesov a aktivít virtuálneho stroja bez nutnosti inštalácie bezpečnostného softvéru v hostovanom systéme*“. Pamäť host'a je pre hostiteľa obyčajná pamäť. Útočník vnútri host'a ju síce môže prepisovať, ale nemôže zabrániť tomu, aby ju niekto zvonku prečítal, a nemôže vypnúť nič, čo vnútri nebeží. V tomto riešení vnútri sledovaného stroja naozaj nebeží nič zo zberu.

Neplatí, že je to nepriestrelná hranica, a práca si to priznáva: parser potrebuje profil jadra host'a, ktorý sa raz odoberie z host'a (`docs/LIMITACIE.md`, L3); pri niektorých spôsoboch podloženia pamäte čítanie nedotknutej stránky hostiteľovi tú stránku reálne alokuje (L2); pamäť dôverných VM (AMD SEV-SNP, Intel TDX) sa takto prečítať nedá vôbec (L7).

1.3 Čím za to platíme

Cena za pohľad zvonku má meno: **semantic gap**, teda sémantická medzera. Zvonku vidno bajty na fyzických adresách. Nevidno procesy. V snímke pamäte nie je napísané, že na istej adrese začína zoznam procesov a že meno procesu leží istý počet bajtov od jeho začiatku. Tú vedomosť má iba jadro host'a. Ako sa medzera preklenie — profil jadra, hľadanie kotvy, preklad adres, prechod spájanými zoznamami — patrí do kapitoly 5.

1.4 Kde je v tom TCN

Z každej snímky vznikne krátky číselný vektor. Snímky idú za sebou, takže vektory tvoria **časový rad**. Susediace vektory sa poskladajú do **okna** (v tomto kóde 16 snímok) a okno je vstupom modelu.

Konvolúcia je jednoduchá operácia: malé okienko váh sa posúva po postupnosti a v každej polohe spočíta jedno číslo z niekoľkých susedných hodnôt. Tým sa učí rozpoznať krátky vzorec bez ohľadu na to, kde v rade sa vyskytne. **Kauzálna** konvolúcia sa doplní nulami iba zľava, takže výstup v čase t závisí výhradne od vstupov do času t — model nemôže nakuknúť do budúcnosti. **Dilatovaná** konvolúcia preskakuje vstupy s rastúcim krokom (1, 2, 4, ...), čím sa dosah rýchlo predĺži bez toho, aby sa model nafúkol. To je celý TCN. Implementácia je v `tcn/model.py`.

Podľa zadania má TCN slúžiť „pre detekciu anomálií a klasifikáciu aktivít“. Tu treba povedať rovno, čo bude podrobne v kapitolách 6 a 9: cesta dát je zostavená a beží, **model natrénovaný nie je**. Korpus reálnych malvérových vzoriek neexistuje a pripravené syntetické scenáre sa na tréning vedome nepoužili, lebo model natrénovaný na nich by klasifikoval šesť skriptov, ktoré napísal autor práce, nie malvér (docs/LIMITACIE.md, L16). Číslo, ktoré dnes z modelu vyjde, preto nie je detekcia.

1.5 Mapa dokumentu

Mapa je na konci písania prepísaná podľa skutočného členenia, takže kapitola, ktorú tu nájdeš, v texte naozaj je.

kapitola	čo v nej je
1	zadanie, problém, VMI, kde je TCN (táto kapitola)
2	virtualizácia a hypervízory: typy 1 a 2, KVM, memsloty, diery v pamäti, prečo eBPF
3	zberač vmicollect: eBPF programy, hľadanie VM, čítanie stránok, štyri opravy verifikátora, formát .vmicd, plánovač, pôvod kódu
4	ostatné časti zberača: C API vmic.h, hook ABI, backend file, obnova reťazca, retencia, konfiguračné rozhranie -o
5	semantic gap a guestparse: profil jadra, kotva a posun, preklad adries, tabuľky stránok, rekonštrukcia procesov, modulov a soketov, validácia, návratové kódy
6	príznačky a model: per-bin vektor, 22-stĺpcový vektor, okná, normalizácia a leakage, TCN, baseliny, čo sa netvrdí
7	ako sa to preloží a spustí: požiadavky, make, testy, profil, root_run.sh, čo pripraviť na živú ukážku
8	namerané čísla: prostredie merania, zber, rekonštrukcia, optimalizácia hashu, latencie, čo nemáme
9	obmedzenia L1 až L17, nevyriešené hlásenie libbpf a zmazaná vetva zberu korpusu
10	HONESTY.md P1–P12, strojová kontrola check_claims.sh, zakázané čísla z hypTcn002 a prečo sa projekt začal nanovo
11	stav voči zadaniu, nezávislý posudok a otázky na obhajobe
12	slovníček pojmov

2. Virtualizácia, hypervízor a kde v tom sedí náš nástroj

2.1 Virtuálny stroj a hypervízor

Virtuálny stroj je počítač, ktorý existuje iba ako program. Operačný systém vnútri neho si myslí, že má vlastný procesor, vlastnú RAM a vlastný disk. V skutočnosti ich dostáva pridelené od hostiteľského stroja.

Hypervízor je vrstva, ktorá tento klam udržiava. Rozdeľuje skutočný hardvér medzi virtuálne stroje a dbá na to, aby jeden hosť nevidel do pamäte druhého.

Delia sa na dva typy:

typ	kde beží	príklady
Typ 1 („bare-metal“)	priamo na hardvéri, pod ním nie je iný OS	Xen, VMware ESXi, Hyper-V
Typ 2 („hosted“)	ako program v bežnom operačnom systéme	VirtualBox, VMware Workstation

KVM sa do tohto delenia vtesnáva zle a pre túto prácu je to dôležité. Nie je to samostatný systém ako Xen, ale ani obyčajný program ako VirtualBox. Je to **modul jadra Linuxu** — kus kódu zavedený priamo do jadra hostiteľa, ktorý z toho jadra urobí hypervízor. Preto sa oň opierame: čo KVM vie o host'ovi, to vie **jadro hostiteľa**, a do jadra hostiteľa sa vieme dostať.

Že je na tomto stroji KVM naozaj modul, a nie priamo zabudovaný do jadra, vypíše `lsmod`:

```
$ lsmod | grep ^kvm
kvm_intel          548864    4
kvm                1568768    3 kvm_intel
```

kvm je spoločná časť, `kvm_intel` je časť pre virtualizačné inštrukcie procesorov Intel (na AMD by tam bolo `kvm_amd`). Pri jadre preloženom s `CONFIG_KVM=y` by `lsmod` nevypísal nič; kód s tým počíta (komentár pri `btf__parse_split` v `vmicollect/src/backend_ebpf.c`).

2.2 KVM a QEMU: kto robí čo

KVM sám o sebe virtuálny stroj nespustí. Vie len to, čo sa bez privilégií jadra spraviť nedá: prepnúť procesor do režimu, v ktorom host'ov kód beží priamo na kremíku, a zachytiť okamih, keď host' siahne na niečo, čo mu nepatrí. Nevie nakresliť obrazovku, nevie emulovať sieťovú kartu ani disk.

To robí **QEMU** — obyčajný proces v používateľskom priestore. QEMU otvorí `/dev/kvm`, požiada o vytvorenie virtuálneho stroja, alokuje pre host'a pamäť a emuluje zariadenia. Keď host' zapíše na port grafickej karty, KVM beh host'a zastaví a odovzdá udalosť QEMU.

Na tomto hostiteľovi je to jeden konkrétny proces:

```
$ ps -eo pid,comm,args | grep qemu
 65607 qemu-system-x86_64 /usr/bin/qemu-system-x86_64 -name guest=hyptcn-guest ... -
machine pc-q35-10.1
$ ls -l /proc/65607/fd | grep kvm
14 -> /dev/kvm
15 -> anon_inode:kvm-vm
28 -> anon_inode:kvm-vcpu:0
30 -> anon_inode:kvm-vcpu:1
```

Čo je „anonymný inode“. Inode je v Linuxe záznam o súbore. Jadro však potrebuje aj „súbory“, ktoré na disku nie sú a nemajú meno v adresári — napríklad rukoväť na otvorený virtuálny stroj. Vyrobí si preto inode bez miesta v súborovom systéme a proces ho vidí ako odkaz v tvare `anon_inode:<meno>`. To meno je jediné, čo sa dá zvonku prečítať, a KVM doň zapisuje `kvm-vm`, resp. `kvm-vcpu:<číslo>`.

Deskriptor `anon_inode:kvm-vm` je teda rukoväť na virtuálny stroj vnútri jadra. Práve jeho hľadá náš zberač (`pick_vm` v `backend_ebpf.c`, funkcia `vmic_probe` v `vmicollect/bpf/vmic_kvm.bpf.c`) — je to spoľahlivejší znak „tento proces je virtuálny stroj“ než meno procesu. Deskriptory `kvm-vcpu:0` a `kvm-vcpu:1` zodpovedajú dvom virtuálnym procesorom.

2.3 Pamäť host'a je v skutočnosti pamäť procesu QEMU

Toto je jediná vec, na ktorej stojí celý projekt.

Keď QEMU štartuje virtuálny stroj s 2 GiB RAM, nedostane 2 GiB fyzických čipov. Zavolá `mmap()` a vypýta si od jadra hostiteľa 2 GiB obyčajnej anonymnej pamäte — presne tak, ako by si ju vypýtal ktorýkoľvek iný program. Potom cez rozhranie `KVM_SET_USER_MEMORY_REGION` povie KVM: „tento kus mojej pamäte je pre host'a fyzická RAM od adresy nula“.

Čo je `mmap`. Systémové volanie, ktorým proces požiada jadro, aby mu do jeho adresného priestoru vložilo oblasť pamäte. Môže to byť obsah súboru, alebo čistá prázdna pamäť (vtedy sa hovorí **anonymné** mapovanie). Od tej chvíle proces s tou oblasťou pracuje ako s obyčajným poľom v pamäti, bez ďalších systémových volaní. `mmap` sa v tomto dokumente objaví ešte raz, v časti 3.4, kde si ním zberač do svojho adresného priestoru vloží BPF mapu.

Dôsledok: **RAM host'a je bežná pamäť bežného procesu.** Existuje pod adresami v procese QEMU a dá sa čítať tými istými prostriedkami ako pamäť ktoréhokoľvek iného procesu.

Vidno to priamo. V mapách procesu 65607 je jediné dvojgigabajtové anonymné mapovanie:

```
$ awk '...' /proc/65607/maps
7fb801000000-7fb881000000 2147483648 2.000 GiB rw-p (bez súboru)
```

A keď v tej istej chvíli spustíme `sudo scripts/root_run.sh probe`, zberač vypíše, kde host'ova pamäť začína:

```
slot 0: GPA 0x000000000000-0x00000009ffff HVA 0x7fb801000000
slot 8: GPA 0x000000100000-0x000007ffffff HVA 0x7fb801100000
```

Adresa `0x7fb801000000` je presne začiatok toho anonymného mapovania. Fyzická adresa 0 v host'ovi je bajt na adrese `0x7fb801000000` v procese QEMU. To nie je tvrdenie z literatúry, to je porovnanie dvoch výpisov z toho istého stroja z toho istého behu (`data/results/probe_20260919T142627Z.json`).

2.4 Stránka, gfn, GPA, HVA, GVA a čo je memslot

Stránka je najmenší kus pamäte, s ktorým procesor pri mapovaní pracuje. Na `x86_64` má **4096 bajtov** (4 KiB). Pamäť sa nemapuje po bajtoch, ale po stránkach, takže aj všetky mapy, tabuľky a štatistiky v tejto práci sú po stránkach. Doména `hyptcn-guest` má 2 164 396 032 B RAM, čo je presne **528 417 stránok** (vlastný výpočet `2164396032 / 4096`; veľkosť RAM je z `probe_20260919T142627Z.json`, kľúč `values.ram_bytes`).

gfn (guest frame number) je poradové číslo stránky vo fyzickej pamäti host'a. Je to jednoducho fyzická adresa host'a vydelená 4096. Keď teda v štruktúre `memslotu` uvidíš pole `base_gfn`, znamená to „začiatok tohto bloku, vyjadrený v stránkach“, nie v bajtoch. Jadrá ukladajú rozsahy v stránkach preto, aby sa do 64-bitového čísla zmestilo o 12 bitov viac adresného priestoru a aby sa nedala zadať adresa, ktorá nie je zarovnaná na stránku.

Tri skratky, ktoré sa ďalej používajú neustále:

skratka	plným menom	po slovensky
GVA	Guest Virtual Address	virtuálna adresa vnútri host'a — to, čo vidí program bežiaci v host'ovi
GPA	Guest Physical Address	fyzická adresa host'a — to, čo host' považuje za adresu na RAM čipe
HVA	Host Virtual Address	virtuálna adresa v procese QEMU na hostiteľovi

Reťaz je teda `GVA → GPA → HVA`. Prvý krok robí jadro host'a cez svoje tabuľky stránok (rieši ho `guestparse`, kapitola 5). Druhý krok je predmet tejto kapitoly.

Preklad `GPA → HVA` nie je vzorec. Je to **tabuľka**, ktorú si KVM drží v štruktúre `struct kvm`, a jej položky sa volajú **memsloty** (`struct kvm_memory_slot`). Jeden `memslot` hovorí: „rozsah fyzických adries host'a od tejto po tamtú leží v procese QEMU na tejto adrese“. Nič viac, nič menej. Náš BPF program z každého slotu číta polia `base_gfn` (začiatok v stránkach), `npages` (dĺžka v stránkach), `userspace_addr` (HVA), `flags` a `id`; ich pozície v štruktúre dostane za behu z ladiacich informácií jadra (BTF), nie natvrdo z prekladu — zoznam je v tabuľke `KFIELDS` v `backend_ebpf.c`.

Zberač vie spracovať najviac 64 `memslotov` (`VMIC_BPF_MAX_SLOTS` v `vmicollect/bpf/vmic_bpf_abi.h`). Meraná doména `hyptcn-guest` ich má 10 vo všetkých štyroch uložených behoch `probe`. Koľko `memslotov` má „bežná“ VM, táto práca nemerala a žiadny artefakt v repozitári to nehovorí, takže sa to v texte netvrdí.

2.5 Prečo nestačí `process_vm_readv`

Ak je pamäť host'a len pamäť procesu, prečo ju jednoducho neprečítať volaním `process_vm_readv()`, ktoré Linux ponúka na čítanie cudzieho procesu?

Lebo bajty sú len polovica. Dostali by sme obsah, ale nie **význam**. Zvonka sa nedá zistiť, ktorá časť adresného priestoru QEMU je ktorá fyzická adresa host'a. Bez tejto mapy by sme hádali z `/proc/<pid>/maps`, kde stojí len „tu je 2 GiB anonymnej pamäte“ — nie, že ide o GPA 0 až 2 GiB, a už vôbec nie, čo sú tie drobné kúsky pod hranicou 1 MiB.

Túto mapu drží modul `kvm` vo svojich `memslotoch` a von ju žiadnym rozhraním neposkytuje. QEMU ju pozná, lebo ju samo zapísalo, ale pýtať sa jeho znamená pýtať sa monitora na jeho vlastný pohľad na seba a potrebovať k tomu spolupracujúci management stack.

Dôležitá poctivosť, ktorú treba vedieť povedať komisii: samotné kopírovanie bajtov je v oboch cestách **to isté**. BPF program používa `bpf_copy_from_user_task()`, čo je pod kapotou `access_process_vm()` — presne to, čo volá `process_vm_readv()`. Prínos eBPF nie je v rýchlejšom alebo hlbšom čítaní. Prínos je výhradne tá mapa. (Podrobne v `docs/ARCHITEKTURA.md`, časť 1.2.)

2.6 Prečo v pamäti host'a sú diery

Fyzický adresný priestor počítača architektúry x86 nie je súvislý pás RAM. Historicky a kvôli hardvéru sú v ňom vynechané okná: **VGA diera** na `0xA0000`, priestor pre BIOS, a veľké **PCI hole** pod hranicou 4 GiB.

Čo v tých oknách je. Nie RAM, ale **MMIO** (memory-mapped I/O). Zariadenie ako sieťová karta alebo grafická karta má svoje riadiace registre, a namiesto zvláštnych inštrukcií sa k nim pristupuje ako k pamäti: procesor zapíše na adresu a zariadenie ten zápis zachytí. Rozsah adries, ktorý si zariadenia na tento účel rezervujú, sa volá **PCI hole** — z pohľadu RAM je to diera, z pohľadu zariadení je to ich adresné okno. Keby sme z tých adries čítali, dostaneme chybu alebo nezmysel, nie obsah pamäte.

Naša VM to ukazuje presne. Má 2,02 GiB RAM (2 164 396 032 B), ale jej fyzický priestor siaha po 4 GiB (max GPA `0x100000000`). Tých 2,02 GiB je rozprestretých v 10 memslotoch. Keď susediace zlúčime, vznikne **5 efektívnych oblastí** (tak ich počíta aj `vmicollect/src/meta.c` a tak ich hlási aj samotný beh: zbierane oblasti: 5, spolu 2.02 GiB) a medzi nimi **4 diery** v celkovej veľkosti 1,984 GiB:

oblasti (GPA)	veľkosť	diery (GPA)	veľkosť
<code>0x00000000–0x000009ffff</code>	640 KiB	<code>0x0000a0000–0x0000bffff</code>	128 KiB (VGA)
<code>0x0000c0000–0x07ffffff</code>	2047,25 MiB	<code>0x080000000–0x0faffffff</code>	1968 MiB (PCI hole)
<code>0x0fb000000–0x0fbffffff</code>	16 MiB	<code>0x0fc000000–0x0fedffffff</code>	46 MiB
<code>0x0fee00000–0x0fee00fff</code>	4 KiB	<code>0x0fee01000–0x0fffbffff</code>	17,75 MiB
<code>0x0fffc0000–0x0fffffff</code>	256 KiB		

(Vlastný výpočet nad `data/results/probe_20260919T142627Z.json`, kľúč `values.memslot_ranges`; súčet oblastí je 2 164 396 032 B, čo sedí na bajt s hlásenou veľkosťou RAM. Súčet dier je 2 130 571 264 B = 1,984 GiB.)

Keďže memsloty presne hovoria, čo je namapované, zberač zbiera **iba oblasti**. Diera sa nečíta ani nehashuje, a čo je podstatnejšie, neobjaví sa v štatistikách: `read_errors` je potom naozaj počet chýb a `changed_ratio` sa počíta zo skutočnej RAM, nie z adresného priestoru.

Čo sa stane, keď sa oblasti zadajú ručne a niektorá dieru pretína. Vtedy `vmic_read` vráti `KIND_HOLE` aj s dĺžkou diery a zberač ju vyplní nulami a pokračuje — diera sa preskočí celá, nie tak, že by sa čítanie na nej zastavilo a zahodilo aj pamäť za ňou. **Ako veľmi je to overené, treba povedať presne**, lebo to je jedno z miest, kde sa dá ľahko tvrdiť viac, než platí. Regresný test `vmicollect/tests/hole_test.c` beží nad **falošným (fake) backendom**, ktorý je v samotnom súbore testu a má úmyselne najhoršiu prípustnú sémantiku. Testuje sa ním **kontrakt funkcie `read_pa()` a spoločná čítacia slučka zberača**, nie vetva `KIND_HOLE` v BPF programe. Tá vetva v testoch **nikdy nebežala** (`docs/LIMITACIE.md`, L5; `HONESTY.md`, P5). Jediné, čo o nej vieme z reálnych behov, je, že počet chýb čítania nad doménou `hyptcn-guest` bol nulový. Výstup testu z vlastného behu make `-C vmicollect test dnes:`

```
build/hole_test
chunks=2 read_errors=1 filled_zero=131072 bytes_read=1966080
OK: pamat za dierou sa zachovala, obraz je zhodny s referenciou
```

2.7 Kde v tomto obrázku sedí náš nástroj

Zberač je rozdelený rovnako ako samotné KVM/QEMU — časť v jadre, časť v používateľskom priestore:

```
hostiteľ, jadro 7.1.13-100.fc43.x86_64
-----
JADRO HOSTITEĽA
modul kvm --> struct kvm --> tabuľka memslotov (GPA -> HVA)
```

```

      ^
      | (2) prečíta memsloty a skopíruje stránky
      |
programy eBPF: vmic_probe , vmic_read      (vmic_kvm.bpf.c)
      |
      | (3) stránky cez BPF mapu 'pages'
      v
-----
POUŽÍVATEĽSKÝ PRIESTOR

proces qemu-system-x86 <----- (1) nájdi proces VM
    2 GiB anonymné mapovanie      vmicollect
    = RAM host'a                  backend_ebpf.c
                                   - nájde fd 'kvm-vm'
                                   - z BTF vyčíta offsety polí
                                   - volá BPF programy
                                   - zapíše snímku .vmicd

```

Používateľská časť (backend_ebpf.c) nájde proces monitora a deskriptor kvm-vm a z BTF prečíta offsety polí. Jadrová časť — dva programy eBPF — z deskriptora vytiahne struct kvm, prejde hašovaciu tabuľku memslotov a skopíruje žiadané stránky do prenosového okna, odkiaľ si ich používateľská časť prevezme.

Prečo sa tomu dá hovoriť „modul na úrovni hypervízora“. Nie preto, že beží v jadre; v jadre beží každý program eBPF. Dôvod je, čo číta: vnútorné dátové štruktúry práve modulu kvm, teda tabuľku memslotov. To je pohľad hypervízora na host'a, nie pohľad host'a na seba. A host' o tom nevie — vnútri neho nebeží žiadny agent, ktorý by sa dal vypnúť.

Čím to nie je. Nie je to zavedený modul jadra (.ko) ani patch hypervízora — jadro ani KVM sa nemenia. Nezachytáva udalosti VM: programy sú SEC("syscall"), nie sú zavesené na žiadny kprobe ani tracepoint, spúšťa ich zberač v termíne plánovača. A je to rozhranie **pozorovacie, nie riadiace**: zastaviť VM nevie (.pause = NULL), takže snímka je živá — každá stránka je konzistentná sama v sebe, ale dve stránky môžu byť z rôznych okamihov (limitácia L1).

Čo to stojí. Root na hostiteľovi (CAP_BPF, CAP_PERFMON), jadro preložené s CONFIG_DEBUG_INFO_BTF a závislosť od vnútorného rozloženia štruktúr kvm. Neinvazívnosť má výnimku: číta sa cez adresný priestor QEMU, takže pri non-anonymne podloženej pamäti sa nedotknutá stránka jej prečítaním na hostiteľovi reálne alokuje (L2). Pri našej VM to hlási samotný probe — 4 KiB pamäte host'a je podložených cez /dev/zero (deleted) a nástroj na to varuje (varovanie je v stderr uložených artefaktov probe_20260918T154840Z.json, run_20260919T110017Z.json aj run_20260919T112521Z.json).

Poznámka k pôvodu zberača. Táto otázka má v dokumente jednu jedinú odpoveď a je v časti 3.9. Formulácia „prevzatý kód neznámeho autorstva“, ktorá je ešte v README.md, docs/ARCHITEKTURA.md 1.2 a HONESTY.md P9, je nepresná a opravuje sa; do textu práce sa preberať nemá.

3. Zberač: ako sa dostaneme k pamäti bežiaceho stroja

3.1 Čo je eBPF a prečo ho jadro kontroluje

Operačný systém má dve oddelené časti. **Jadro** (kernel) je program, ktorý vlastní hardvér a pamäť všetkých ostatných programov. **Používateľský priestor** je všetko ostatné — prehliadač, QEMU, náš zberač. Bežný program sa do jadra nedostane; môže ho len požiadať o službu cez systémové volanie.

eBPF je výnimka. Je to spôsob, ako do bežiaceho jadra vložiť malý vlastný program a nechať ho tam bežať. Je to ako zásuvný modul do motora, ktorý ide za jazdy — a presne preto existuje problém: keby sa dal vložiť ľubovoľný kód, každá chyba v ňom by zhodila celý stroj.

Riešením je **verifikátor**. Je to časť jadra, ktorá program pred prijatím prejde inštrukciou po inštrukcii a *dokáže*, že skončí a že nesiahne mimo dovolenej pamäte. Nič nespúšťa — simuluje všetky možné cesty programu. Odtiaľ plynie jeho prísnosť: cyklus musí mať dokázateľnú hornú hranicu, každý ukazovateľ musí mať dokázateľný rozsah, a celkový počet preskúmaných inštrukcií má tvrdý strop 10^6 . Keď dôkaz nevyjde, jadro program odmietne a nenačíta ho. Toto nás stálo štyri opravy (časť 3.5).

Čo je „BPF mapa“. Program eBPF beží v jadre, ale niekto v používateľskom priestore ho musí niečím kŕmiť a niekto od neho musí výsledok prevziať. Na to slúži **BPF mapa**: dátová štruktúra, ktorú vlastní jadro, do ktorej program v jadre zapisuje a číta z nej, a ktorú súčasne vidí aj používateľský proces cez deskriptor súboru. Je to spoločná schránka s dvoma kľúčmi. Mapy majú typy (pole, hašovacia tabuľka, kruhový buffer) a veľkosť sa určuje pri vytvorení. V tomto projekte sú nosné dve: `koff` (offsety polí jadrových štruktúr, tečie z používateľského priestoru do jadra) a `pages` (prenosové okno pre skopírované stránky, tečie opačne).

Prečo vôbec eBPF, keď pamäť host'a je len pamäť procesu QEMU? Bajty by sa dali vytiahnuť aj zvonka cez `process_vm_readv()`. Zvonka sa však nedá zistiť to podstatné: **ktorá časť adresného priestoru QEMU je ktorá fyzická adresa host'a**. Túto mapu drží modul `kvm` vnútri jadra a von ju cez žiadne rozhranie neposkytuje. Prínos eBPF je výhradne tá mapa — v kopírovaní bajtov sú obe cesty rovnocenné.

3.2 Dva programy, ktoré nespúšťa nikto okrem nás

V súbore `vmicollect/bpf/vmic_kvm.bpf.c` sú dva programy:

program	čo robí
<code>vmic_probe</code>	nájde v procese VMM <code>struct kvm</code> a prejde tabuľku <code>memslotov</code> → mapa GPA ↔ HVA
<code>vmic_read</code>	preloží fyzickú adresu host'a na adresu v QEMU a skopíruje stránky do prenosového okna

Obidva sú označené `SEC("syscall")`. Bežný BPF program sa *zavesí* na udalosť — na systémové volanie, na tracepoint, na sieťový paket — a jadro ho spustí vtedy, keď udalosť nastane. Tieto dva nie sú zavesené na nič. Zberač si ich volá sám, cez `bpf_prog_test_run_opts()` (funkcia `run_prog()` v `vmicollect/src/backend_ebpf.c`). **Čo to volanie je:** rozhranie `libbpf`, ktorým používateľský proces povie jadru „spusti mi tento načítaný BPF program teraz, s týmto vstupným buffrom, a vráť mi návratovú hodnotu“. Napriek názvu to nie je len testovacie rozhranie; je to jediný spôsob, ako si BPF program zavolať *vtedy, keď chceme my*, bez toho, aby sme ho na niečo vešali.

Je to dôležité a dá sa to povedať jednou vetou: **na horúcich cestách KVM nevisí nič**. Keby bol program zavesený na tracepoint v KVM, platila by zaň každá jedna operácia virtuálneho stroja, aj keď práve nezbierame. Takto bežiaci VM o zberači nevie a náklad vzniká len v tých milisekundách, keď má plánovač termín.

Program okrem toho nepozná rozloženie ani jednej jadrovej štruktúry. Všetky bajtové offsety dostane v BPF mape `koff(struct vmic_bpf_koff v vmicollect/bpf/vmic_bpf_abi.h)` a zberač ich vyčíta z BTf — popisu, ktorý má jadro samo o sebe v `/sys/kernel/btf/vmlinux` a `/sys/kernel/btf/kvm`. Keď pole v novom jadre zmizne, zberač to povie menom namiesto tichého čítania smetí.

3.3 Ako sa nájde virtuálny stroj

Reťaz je krátka a každý článok má meno:

1. Každý proces, ktorý má otvorenú VM, drží **deskriptor súboru** (číslo, pod ktorým proces pozná otvorený súbor) ukazujúci na **anonymný inode** s menom `kvm-vm` (čo je anonymný inode, hovorí časť 2.2). Vyrába ho KVM volaním `anon_inode_getfile("kvm-vm", ...)`. Je to spoľahlivejší znak než meno procesu — funguje pre QEMU aj `crosvm` a nepotrebuje `libvirt`.
2. Zberač si v používateľskom priestore prejde `/proc/<pid>/fd` a hľadá symbolický odkaz `anon_inode:kvm-vm` (funkcia `proc_kvm_fd()`). Číslo, ktoré nájde, pošle programu ako `fd_hint`.
3. V jadre `vmic_probe` cez tento tip prečíta `struct file` a z jeho poľa `private_data` dostane `struct kvm` — riadiacu štruktúru jedného virtuálneho stroja. Keď tip nesedí, prehľadá tabuľku deskriptorov (strop `VMIC_BPF_MAX_FDS = 1024`).
4. Z `struct kvm` vedie cesta na `kvm_memslots` a jej hašovaciu tabuľku `id_hash` (128 bucketov). Každá jej položka je **memslot**: súvislý kus fyzického priestoru host'a (`base_gfn`, `npages`) namapovaný na adresu v QEMU (`userspace_addr`).

Čo je hašovacia tabuľka a čo je bucket. Hašovacia tabuľka je spôsob, ako rýchlo nájsť záznam podľa kľúča. Kľúč (tu `id memslotu`) sa prepočíta jednoduchou funkciou na číslo priečinka; priečinkov sa volá **bucket** a tabuľka ich má pevný počet, tu 128. V každom buckete visí krátky **reťazec** (spájaný zoznam) tých záznamov, ktorých kľúče vyšli do toho istého priečinka. Hľadanie je teda: spočítaj bucket, prejdi jeho krátky reťazec. Keď nepoznáme kľúč a chceme všetky záznamy — a to je presne náš prípad, lebo chceme celú mapu memslotov — musíme prejsť všetkých 128

bucketov a v každom celý reťazec. Odtiaľ je aj odhad, ktorý v časti 3.5 zhodil verifikátor: 128 bucketov × reťazec do 8 uzlov × 5 čítaní na uzol.

Čo je RCU a prečo sa bez neho tabuľka čítať nesmie. RCU (Read-Copy-Update) je spôsob, akým jadro Linuxu umožňuje čítať zdieľané dátové štruktúry bez zámku, ktorý by brzdil ostatných. Pravidlo je: kto zapisuje, nemení existujúci uzol, ale vyrobí nový a starý zahodí až potom, keď má istotu, že ho už nikto nečíta. A tú istotu dostane tak, že čitatelia sa ohlásia — vstúpia do **RCU čítacej sekcie**. Kým je čitateľ vnútri, jadro uvoľnenie starých uzlov odloží. Keby náš program čítal tabuľku deskriptorov alebo memslotov bez ohlásenia, mohla by mu spod rúk zmiznúť pamäť, ktorú práve dereferencuje. Preto celý prechod beží pod `bpf_rcu_read_lock()` a `bpf_rcu_read_unlock()`.

`struct kvm` sa medzi behmi nikdy neukladá — hľadá sa nanovo. Pred prechodom a po ňom sa navyše číta pole **generation**: je to počítadlo, ktoré KVM zvýši pri každej zmene tabuľky memslotov. Keď sa medzitým zmenilo, tabuľka bola počas nášho čítania menená, je teda nekonzistentná a cyklus sa radšej zahodí.

Čerstvý beh z 19. 9. (`sudo -A scripts/root_run.sh probe`, uložené v `data/results/probe_20260919T142620Z.json`):

```
domena          : hyptcn-guest
backend         : ebpf/kvm
velkost RAM     : 2.02 GiB
max. fyz. adresa : 4.00 GiB (0x1000000000)
pocet vCPU      : 2
test citania    : 16646144 B za 14.4 ms = 1103 MiB/s
```

Desať memslotov, najväčší z nich GPA `0x1000000-0x7fffffff` na HVA `0x7fb801100000`.

3.4 Ako sa čítajú stránky

`vmic_read` dostane fyzickú adresu host'a, nájde v tabuľke slotov ten pravý, prepočíta adresu a zavolá `bpf_copy_from_user_task()`. Ten ide cez `access_process_vm()` — tú istú cestu, akú používa `process_vm_readv()` — takže číta bez ohľadu na to, či VM práve beží.

Cieľom kopírovania je BPF mapa `pages` s príznakom **BPF_F_MMAPABLE** (2 MiB). Ten príznak hovorí jadru, že táto mapa sa smie dať používateľskému procesu namapovať priamo do jeho adresného priestoru. Zberač to pri štarte aj urobí volaním `mmap` (`backend_ebpf.c`, riadok 787: `p->win = mmap(NULL, p->win_size, PROT_READ, MAP_SHARED, p->map_pages, 0);`). Od tej chvíle je obsah mapy pre zberač obyčajné pole v pamäti — žiadne systémové volanie na stránku.

Vďaka tomu stránky host'a **prekročia hranicu jadro/používateľ presne raz**. Nie je to nulové kopírovanie a tak sa to nazvať nesmie: v používateľskom priestore nasleduje ešte jeden `memcpy` z okna do cieľového buffra (`backend_ebpf.c` riadok 1128; limitácia L8). Presný opis je teda „jedno prekročenie hranice jadro/používateľ a jedno kopírovanie v používateľskom priestore na blok“.

Keď veľké kopírovanie zlyhá, skúsi sa to znovu po jednotlivých stránkach, aby jedna nenamapovaná stránka nezhodila celý blok. Keď adresa nepatrí do žiadneho slotu, program vráti `KIND_HOLE` aj dĺžku diery a zberač ju vyplní nulami — diery sa preskakujú, nie orezávajú. Nakoľko je táto vetva overená, hovorí časť 2.6 a L5: v testoch nikdy nebežala.

3.5 Štyri chyby, ktoré program nepustili do jadra

Toto je najkonkrétnejší kus vlastnej práce. **Pozor na formuláciu pôvodu** — správne znenie je v časti 3.9 a je to: kód vznikol 2026-08-26 vo vlastnom sedení autora s jazykovým modelom, nie je to prevzatý cudzí celok, ale cesta cez eBPF **nikdy nebola overená proti bežiacemu jadru**. Pri prvom pokuse o načítanie na hostiteľovi s jadrom 7.1.13 a libbpf 1.6.1 sa program do jadra nedostal. Štyri nezávislé príčiny, všetky opravené v committe 178ce4f:

(1) BTF: dopredná deklarácia verzus plná štruktúra. Riadok `extern struct task_struct *bpf_task_from_vpid(__s32) __ksym;` bez definície typu vyrobí v BTF objektu druh FWD („taká štruktúra existuje, obsah neviem“), kým jadro má STRUCT. libbpf porovnáva *druh* typu a nezhodu odmieta:

```
extern (func ksym) 'bpf_task_from_vpid': func_proto [33]
incompatible with vmlinux [45837]
```

Čo znamená __ksym. Je to značka v zdrojáku BPF programu, ktorá hovorí: „tento symbol nie je v mojom objekte, nájdeš ho v jadre“. Používa sa pre **kfunkcie** — funkcie jadra, ktoré jadro BPF programom vedome sprístupnilo. Pri načítaní libbpf takto označený symbol vyhľadá v BTF jadra a porovná, či sa typy zhodujú. Práve to porovnanie tu

padlo. Oprava je jednoriadková: `struct task_struct { char __opaque; };`. Druh aj meno sedia, obsah sa nikdy nečíta, takže závislosť na rozložení jadra nevzniká.

(2) Limit zložitosti skokov (8192). Ručný cyklus `for (i = 0; i < 1024; i++)` nad tabuľkou deskriptorov, asi osem vetiev na iteráciu. Verifikátor cyklus rozbalí a počíta skoky: `The sequence of 8193 jumps is too complex` → -E2BIG.

(3) Limit 10^6 inštrukcií pri prechode memslotov. 128 bucketov × reťazec do 8 uzlov × 5 čítaní na uzol (čo sú bucket a reťazec, hovorí časť 3.3): `BPF program is too large. Processed 1000001 insns.`

(4) Limit 10^6 inštrukcií pri záložnom kopírovaní. 512 stránok s vetvením v tele → 42 900 stavov, opäť -E2BIG.

Chyby 2–4 majú jednu príčinu a jedno riešenie. Príčina: verifikátor ručný cyklus **rozbalí** a overuje každú iteráciu zvlášť. Riešenie: `bpf_loop()` — pomocná funkcia jadra, ktorej sa odovzdá telo cyklu ako samostatná funkcia. Verifikátor ho overí **raz** a o počet opakovaní sa postará jadro za behu. V kóde sú to `fd_scan_step()`, `slot_scan_bucket()` a `page_copy_step()`. Cena je, že do callbacku sa nedá preniesť ukazovateľ ani započítaná referencia — preto sa mapy v každom kroku vyhľadávajú znovu a `page_copy_step()` si task berie nanovo cez `bpf_task_from_vpid()` / `bpf_task_release()`. Sémantika sa nezmenila: rovnaké poradie čítaní, rovnaké kontroly generation, rovnaké zaobchádzanie s dierami. Zmenila sa iba forma cyklov.

Bez týchto štyroch opráv zberač nezberá nič. Je to bod, ktorý sa pri otázke „čo z toho ste spravili vy“ dá ukázať na committe aj na chybových hláškach.

3.6 Delta snímky a formát .vmicd

Plná snímka 2,02 GiB každých 5 sekúnd je asi 1,4 TiB za hodinu (vlastný výpočet: $2,02 \text{ GiB} \times 720 \text{ cyklov} = 1454 \text{ GiB}$; je to rádový odhad z veľkosti a periódy, nie meranie). Nepoužiteľné. Preto `writer_delta.c` ukladá **iba zmenené stránky**: pre každú stránku (4 KiB) spočíta rýchly 64-bitový haš a zapíše ju len vtedy, keď sa líši od predchádzajúceho cyklu.

Tri pojmy, ktoré sa odtiaľto ťahajú celým dokumentom:

- **plná snímka** — zapíše sa každá nenulová stránka pamäte. Je to referencia, od ktorej sa počítajú ďalšie rozdiely.
- **delta snímka** — zapíše sa iba tie stránky, ktoré sa od predchádzajúceho cyklu zmenili.
- **reťazec (chain)** — plná snímka a všetky delty za ňou, ktoré sa o ňu opierajú. Reťazec má `chain_id` a snímky v ňom poradové číslo `seq`. Prvá snímka reťazca sa volá **baseline snímka**. Bez nej sa delty za ňou nedajú poskladať späť na obraz pamäte — preto retencia maže po celých reťazcoch (časť 4.5).

Každých `delta_full_every` snímok (predvolene 20) sa zapíše nová plná snímka, ktorá začína nový reťazec — aby sa chyba nešírila a dalo sa začať prehrávať od ľubovoľného miesta.

Formát je zámerne triviálny: hlavička **64 bajtov** little-endian (magic `VMICDLT1`, verzia, veľkosť stránky, id reťazca, poradové číslo, logická veľkosť pamäte, počet záznamov, príznaky, podpis oblastí), za ňou počet záznamov × (u64 index stránky + obsah stránky). Záznamy sú pevne dlhé, takže index sa dá postaviť čítaním samotných indexov. Formát zapisuje `vmicollect/src/writer_delta.c`, číta ho tá istá jednotka (`vmic_delta_restore`) a nezávisle od nej `guestparse/image.py`.

Koľko sa naozaj zmení, je zmerané. Z behu `data/results/run_20260919T112521Z.json` (8 cyklov, perióda 5 s, hodnoty z kľúča `stderr` toho artefaktu):

snímka	typ	zmenené stránky	čítanie	cyklus
#0	plná	89 098 / 528 417 (16,86 %)	1296,6 ms	1630,9 ms
#1	delta	231 (0,04 %)	632,1 ms	639,9 ms
#2	delta	434 (0,08 %)	600,2 ms	609,1 ms
#3	delta	253 (0,05 %)	614,8 ms	624,3 ms
#4	delta	227 (0,04 %)	609,9 ms	618,1 ms
#5	delta	298 (0,06 %)	634,4 ms	639,4 ms
#6	delta	242 (0,05 %)	634,2 ms	642,5 ms
#7	delta	188 (0,04 %)	599,9 ms	606,0 ms

Naprieč **všetkými** sidearmi v repozitári (vlastný sken `data/**/*`.json so schémou `vmicollect/1`, 2026-09-19: 50 delta snímok a 21 plných) je prírastok delta snímky **0,032 % – 1,374 %** stránok; horné hodnoty patria behom so

záťažou v host'ovi, dolné nečinné VM. Pri plnej snímke je nenulových **13,36 % – 26,52 %** stránok (zvyšok sú nuly, ktoré sa nezapisujú).

3.7 Plánovač bez driftu

Naivné „spracuj a potom sleep(5)“ má dve chyby. Perióda sa každý cyklus predĺži o čas spracovania, takže po hodine časová rada neseďí — a analýza sekvencií nad posunutou mriežkou nemá zmysel. A Ctrl-C sa prejaví až po dospaní celej periódy.

vmicollect/src/sched.c počíta **absolútne termíny** na monotónnych hodinách (clock_gettime(CLOCK_MONOTONIC)) a spí do nich cez clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, ...). Chyba sa nekumuluje, lebo ďalší termín je next += interval, nie „teraz + interval“. Keď cyklus periódu presiahne, rozhoduje [schedule].overrun: skip drží pôvodnú mriežku a zmeškané sloty zahodí (správna voľba pre časové rady), catch_up ich dobehne, stretch mriežku posunie. V citovanom behu: cycles_skipped: 0, worst_lateness_s: 0.000.

3.8 Živá snímka

Virtuálny stroj sa počas čítania **nezastavuje**. Backend ebpf nemá operáciu pause — vo vmic_backend_ops_t je .pause = NULL (backend_ebpf.c:1167), pretože eBPF je pozorovacie, nie riadiace rozhranie. Čítanie 2,02 GiB trvá stovky milisekúnd až sekundu a host' medzitým beží: stránka prečítaná na začiatku a stránka prečítaná na konci pochádzajú z rôznych okamihov. Spájaný zoznam jadra sa preto môže počas prechodu roztrhnúť.

Sidecar je JSON súbor, ktorý vzniká vedľa každej snímky a nesie o nej metadáta: časy fáz cyklu, počty stránok, kontrolné súčty, zbierané oblasti a (keď je zapnutý) príznakový vektor. Robí ho vmicollect/src/meta.c, schéma je vmicollect/1.

Vlastný sken repozitára 2026-09-19 (všetky súbory data/**/* .json so schémou vmicollect/1): **79 sidecarov** — 45 v data/results/, 28 v data/raw/ a 6 v data/sessions/. Vo **všetkých 79** je capture.paused = false a capture.pause_ms = 0. (Číslo 14, ktoré uvádza staršia verzia docs/LIMITACIE.md L1, je zastarané; tvrdenie o nulovej pause platí, počet nie.)

Je to limitácia L1 a parser s ňou počíta — každý prechod má strop, detekciu cyklu a príznak truncated.

3.9 Kto zberač napísal

V repozitári je zatiaľ na troch miestach formulácia, že autorstvo vmicollect je neznáme (README.md, HONESTY.md P9, docs/ARCHITEKTURA.md 1.2). **Táto formulácia je nepresná a treba ju opraviť**. Zberač nie je prevzatý cudzí kód. Vznikol 2026-08-26 vo vlastnom sedení autora práce s jazykovým modelom, ktoré začalo vetou „ten periodický zber potrebujem cez eBPF a len pre KVM“. Sedenie zapísalo backend_ebpf.c, vmic_kvm.bpf.c, vmic_bpf_abi.h, Makefile a README, jeho podagenti ďalších päť súborov. Časové značky súborov to potvrdzujú: README.md 13:25:23, vmic_bpf_abi.h 13:26:56, backend_ebpf.c 13:27:33 — sedenie končí 13:31.

Správne znenie je teda jedno a to isté v celom dokumente: zberač je autorova práca vzniknutá s pomocou jazykového modelu; pôvodná cesta cez eBPF nikdy nebola overená proti bežiacemu jadru; overenie plus štyri opravy z commitu 178ce4f sú prácou tejto diplomovej práce. Pri obhajobe je to dôležitý rozdiel: prácou nie je len overenie cudzieho riešenia, ale aj jeho návrh.

4. Ostatné časti zberača, ktoré sa dajú ukázať

Táto kapitola je tu preto, že zadanie žiada aj „**API rozhranie**“ a nezávislý posudok (fable-review.md) ho hodnotí ako FUNGUJE. Je to bod, ktorý sa dá započítať medzi splnené, a v dokumente by nemal chýbať.

4.1 C API vmicollect/include/vmic.h a štyri vrstvy

vmicollect/include/vmic.h je jediné verejné rozhranie zberača (**517 riadkov**, vlastné wc -l dnes; docs/ARCHITEKTURA.md uvádza 456, čo je zastaraný údaj — podľa HONESTY.md P12 platí kód). Modul je v ňom rozdelený na štyri nezávislé vrstvy, ktoré sa navzájom vidia len cez štruktúry s ukazovateľmi na funkcie (vtable):

vrstva	súbory	otázka, ktorú rieši
backend	backend_ebpf.c, backend_file.c	odkiaľ berieme bajty
writer	writer_raw.c, writer_delta.c	kam a v akom tvare ich ukladáme
plánovač	sched.c	kedy presne sa má zbierať
hooky	hooks.c	čo sa má stať po každej snímke

collector.c ich iba spája dokopy. Prínos tohto delenia je konkrétny: backend, ktorý by VM vedel pozastaviť, sa dá doplniť bez zásahu do zberača — preto je v konfigurácii kľúč capture.pause aj meranie pause_ms, hoci ho dnes žiadny backend neimplementuje.

Hlavička definuje:

sekcia	obsah
úrovne logovania, vmic_rc_t	VMIC_OK / VMIC_ERR / VMIC_FATAL / VMIC_STOP
vmic_config_t	celá konfigurácia v jednej štruktúre
vmic_vminfo_t	parametre cieľovej VM vrátane zoznamu skutočne namapovaných oblastí
vmic_stats_t	surové počítadlá jedného čítania
vmic_snapshot_t	záznam o jednej snímke; dostane ho writer, sidecar aj každý hook
vmic_backend_ops_t, vmic_capture()	vrstva backendu
vmic_writer_ops_t	vrstva writera
vmic_sched_run()	plánovač
vmic_hook_api_t, vmic_hooks_*	hooky
vmic_config_*, vmic_meta_write,	ostatné
vmic_retention_apply, vmic_delta_restore	

Rozlíšenie VMIC_ERR a VMIC_FATAL nie je formalita: pri ERR zberač preskočí cyklus a skúsi to o periódu neskôr (VM sa napríklad práve migruje), pri FATAL končí (doména neexistuje, chyba knižnica, plný disk).

4.2 Hook ABI: čo plugin dostane a čo nedostane

Hook je zásuvný modul zberača: zdieľaná knižnica načítaná cez dlopen(RTLD_NOW), ktorá musí exportovať tri symboly:

```
int vmic_hook_init(const vmic_hook_api_t *api, const char *args, void **state);
int vmic_hook_snapshot(void *state, const vmic_snapshot_t *snap);
void vmic_hook_fini(void *state);
```

ABI (application binary interface) je dohoda o tom, ako presne vyzerajú tieto funkcie a štruktúry v preloženom kóde — ktoré polia sú v akom poradí a akej veľkosti. Keď sa zberač a plugin preložia proti rôznym verziám vmic.h, dohoda neplatí, a preto štruktúra nesie číslo abi (dnes 1), ktoré si vmic_hook_init kontroluje.

Celá štruktúra, ktorú plugin pri inicializácii dostane, má **tri polia**:

```
typedef struct {
    unsigned abi; /* VMIC_HOOK_ABI, dnes 1 */
    const vmic_config_t *cfg; /* konfigurácia, iba na čítanie */
    void (*log)(int level, const char *fmt, ...);
} vmic_hook_api_t;
```

Pri každej snímke dostane const vmic_snapshot_t *: poradové číslo, id reťazca, cestu k hotovému súboru na disku, cestu k sidecaru, čas, časy pause_ms / capture_ms / write_ms / total_ms, bytes_logical, bytes_on_disk, pages_total, pages_changed, skutočný príznak paused, zbierané oblasti, SHA-256 a ukazovatele na vmic_vminfo_t a vmic_config_t.

Čo NEDOSTANE (limitácia L9, a treba to vedieť povedať skôr, než sa komisia spýta):

- nedostane `read_pa` ani inú funkciu na čítanie pamäte host'a; v `vmic_hook_api_t` nie je ukazovateľ na backend ani na zberač, jediná služba je `log`;
- nedostane obsah stránok, ani ukazovateľ do buffra, ani mapované okno — `len snap->path`, teda cestu k súboru, ktorý si musí otvoriť sám;
- nedostane príznakový vektor;
- nedostane možnosť ovplyvniť zber: návratová hodnota má dva významy, 0 = v poriadku, nenulová = chyba. Pri `[hooks].strict = true` zberač skončí, inak sa plugin vypne do konca behu a zber pokračuje (`vmic_hooks_fire` v `hooks.c`).

Rozhranie je teda notifikačné. Hook sa dozvie, že snímka existuje a kde leží. Beží synchronne v hlavnej slučke a až po dokončení zápisu snímky, takže nespomaľuje host'a, iba zberač — ale keď trvá dlhšie než perióda, zberač začne zmeškávať sloty.

Dôsledok pre návrh skórovania: skórovací hook, ktorý by mal vracať výstup modelu späť do modulu, nemôže pamäť čítať cez rozhranie hooku. Musí si otvoriť `snap->path` sám (a zaplatiť čítanie z disku), alebo pracovať iba s tým, čo je v `sidecar`. Práve preto je `tcn/score.py` samostatný proces, ktorý sleduje výstupný adresár, a nie plugin.

4.3 Backend file

Druhý backend je `file`: namiesto pamäte bežiackej VM číta obyčajný súbor s obrazom pamäte (`vm.backend=file`, `vm.image_path=...`). Nie je to hračka — stoja na ňom testy aj ukážka hooku, lebo **nepotrebuje root ani bežiacu VM**, takže sa dá spustiť na ktoromkoľvek stroji, kde sa repozitár preloží. Overí naraz konfiguráciu cez `-o`, plánovač, `writer delta` a hook API.

Z `docs/ARCHITEKTURA.md`, časť 3.1 (doslovný výstup uloženého behu, cesta dočasného adresára nahradená `$D`):

```
2026-09-18T16:54:15.545Z INFO file: '$D/mem.raw' otvoreny, 8388608 B
2026-09-18T16:54:15.545Z INFO VM 'mem.raw' [file]: RAM 8.00 MiB, max_paddr 0x800000, vCPU 0, strankovanie
2026-09-18T16:54:15.545Z INFO hooks: nacistany './vmicollect/build/example_hook.so' s argumentmi: $D/index
2026-09-18T16:54:15.549Z INFO #0 PLNA 68.00 KiB/8.00 MiB stranky 16/2048 (0.78 %) pauza 0.0 ms citanie
2026-09-18T16:54:16.548Z INFO #1 delta 4.00 KiB/8.00 MiB stranky 0/2048 (0.00 %) pauza 0.0 ms citanie 2
2026-09-18T16:54:17.552Z INFO koniec: 3 snimok, 0 chyb, 0 zmeskaných slotov, najvacsie meskanie 0.000 s, s
```

A súbor, ktorý zapísal plugin (`$D/index.csv`) — to je presne to, čo hook z rozhrania naozaj videl:

```
seq,timestamp_unix,path,bytes_on_disk,pause_ms,capture_ms,total_ms,pages_changed,pages_total
0,1789750455.545,$D/snap/mem.raw_000000_20260918T165415545Z.vmicd,69632,0.000,3.146,3.305,16,2048
```

Pozor na dve veľkosti, ktoré si netreba zamieňať: prvá snímka je plná (16 nenulových stránok z 2048, súbor 65 728 B = 64 bajtov hlavičky + 16 × 4104). Ďalšie dve sú delty s nula zmenenými stránkami: súbor má presne 64 bajtov, teda iba hlavičku — ale `bytes_on_disk` je pri nich 4096, lebo to je najmenšia alokácia na súborovom systéme.

4.4 Obnova reťazca delta snímok

Delta snímka sama osebe obraz pamäte nie je. Podpríkaz `restore` z reťazca poskladá plný obraz:

```
restore --dir D --out F [--chain ID] [--until SEQ]
```

Funkcia je `vmic_delta_restore` a používa ju aj `selftest`. Práve preto sa nesmie zmazať plná snímka bez svojich `delt` (časť 4.5) — bez `baseline` sa zvyšok reťazca obnoviť nedá. Overuje to `make -C vmicollect test`, ktorý nad 16 MiB syntetickým obrazom urobí plnú snímku a tri delty, reťazec obnoví a porovná bajt po bajte. Vlastný beh dnes:

```
2026-09-19T14:50:22.978Z INFO restore: hotovo -> build/selftest/restored.raw (16.00 MiB, 2057 zapisanych
2026-09-19T14:50:22.986Z INFO selftest: OK - obnoveny obraz je bajt po bajte zhodny
2026-09-19T14:50:23.013Z INFO selftest: OK - raw snimka ma spravny SHA-256
2026-09-19T14:50:23.013Z INFO selftest: VSETKO PRESLO. Data su v build/selftest
```

4.5 Retencia

Periodický zber zaplní disk skôr, než sa stihne niečo zmerať. `vmicollect/src/retention.c` preto staré snímky maže podľa troch nezávislých limitov v sekcii `[retention]`: `max_snapshots`, `max_bytes`, `max_age_s` (0 = vypnuté).

Jedna vec, ktorá na tom nie je zrejma: pri delta zbere sa **nesmie** zmazať plná snímka bez jej delt, lebo zvyšok reťazca by sa už nedal obnoviť. Preto sa maže **po celých reťazcoch**, nie po jednotlivých súboroch. Pri raw zbere je každý súbor sám sebe reťazcom, takže ten istý kód funguje pre oboje.

Z toho plynie priznané obmedzenie **L11**: retencia pracuje nad celým output.dir a nerozlišuje domény. V kóde to vidno na tom, že vmic_retention_apply() dostane iba konfiguráciu a id chráneného reťazca, nie identitu domény. Keď do jedného adresára zbierajú dva zberače, mažu si navzájom snímky — každej doméne treba dať vlastný output.dir. Namerané výsledky tejto práce to neovplyvňuje, lebo zbierala jedna doména.

4.6 Konfiguračné rozhranie: -o sekcia.kluc=hodnota

Konfigurácia je jedna štruktúra vmic_config_t; načítava ju config.c z INI súboru (-c subor.conf) a prepínač -o sekcia.kluc=hodnota prebíja súbor po jednotlivých kľúčoch. Prepínač sa dá opakovať. Práve tento zápis vidno vo všetkých uložených príkazoch v data/results/*.json, napríklad:

```
vmicollect run -o vm.domain=hyptcn-guest -o output.writer=delta -o output.hash=sha256 \
-o output.dir=... -o schedule.interval_s=5 -o schedule.max_cycles=10
```

Sekcie a kľúče:

sekcia	kľúče
[vm]	domain, backend, bpf_object, image_path
[schedule]	interval_s, jitter, overrun, max_cycles, duration_s, align
[capture]	pause, pause_max_ms, chunk_size, regions, skip_read_errors, max_read_errors
[output]	dir, writer, name_template, sparse, post_compress, hash, sidecar, delta_page_size, delta_full_every
[features]	enable, bin_bytes, entropy
[retention]	max_snapshots, max_bytes, max_age_s
[hooks]	load (až 16×), strict
[log]	level, file, json

Východzie hodnoty, ktoré tento dokument spomína: schedule.interval_s = 5, output.writer = raw, output.hash = sha256, output.sidecar = true, output.delta_full_every = 20, features.bin_bytes = 16 MiB.

Validácia je prísna a hlási zmysluplne, napríklad:

```
$ ./vmicollect/build/vmicollect run -o output.delta_full_every=0 ...
2026-09-18T16:31:13.882Z ERROR output.delta_full_every musí byť aspoň 1
```

Úplný zoznam kľúčov vypíše vmicollect config --keys, efektívnu konfiguráciu vmicollect config.

5. Semantic gap: ako z bajtov spravíme procesy

5.1 Čo je semantic gap

Snímka pamäte je súbor. Keď ho otvoríme, nie sú v ňom procesy ani moduly — sú v ňom bajty na fyzických adresách. Fyzická adresa je poradové číslo bajtu v RAM host'a, presne ako číslo riadku v obrovskom zošite.

Vnútri host'a pritom procesy existujú. Jadro Linuxu si o každom bežiacom procese drží jednu štruktúru task_struct a všetky ich navliekol na zoznam. Lenže to, **kde** ten zoznam začína a **koľko bajtov od začiatku štruktúry** leží meno procesu, v pamäti nikde napísané nie je. Kompilátor si to zapamätal pri preklade jadra a do výsledku to nezapísal.

Tomuto rozdielu — hypervízor vidí bajty, host' vidí objekty — sa hovorí **semantic gap**. Preklenúť ho znamená dodať zvonku dve chýbajúce informácie: adresy a offsety.

Konkrétne čísla z tohto projektu (vlastný beh dnes nad profilom v repozitári): v host'ovi hyptcn-guest (Debian 12, jadro 6.1.0-42-cloud-amd64) leží pole comm, teda meno procesu, **2976 bajtov** od začiatku task_struct. Celá

štruktúra má **9728 bajtov** (`btbf.txt`, riadok [191] `STRUCT 'task_struct' size=9728 vlen=242`). Pole `tasks`, cez ktoré sú procesy navlečené na zoznam, je na offsete **2192**. Overiteľné:

```
$ python3 -c "from guestparse.profile import Profile; \
    p=Profile.from_dir('profiles/debian12-6.1.0-42-cloud-amd64'); \
    print(p.member('task_struct','comm'), p.member('task_struct','tasks'))"
2976 2192
```

Kód, ktorý túto prácu robí, je balík `guestparse/`, predovšetkým `guestparse/view.py` (trieda `GuestView`) a `guestparse/profile.py` (trieda `Profile`). Zberač `vmicollect` semantiku host'a nepozná vôbec a ani ju poznať nepotrebuje — ten iba kopíruje stránky.

5.2 Profil jadra host'a: päť súborov, dve rôzne doby platnosti

Chýbajúce informácie sa berú z **profilu jadra host'a**, adresár `profiles/debian12-6.1.0-42-cloud-amd64/`. Nie je to mágia ani hádanie: rovnaký vstup potrebujú aj `LibVMI` a `Volatility`.

Adresár má **päť položiek** (vlastný `ls -la` dnes):

súbor	veľkosť	čo v ňom je
<code>kallsyms.txt</code>	4 229 614 B, 98 695 riadkov	adresy symbolov bežiaceho jadra host'a
<code>btbf.txt</code>	9 049 370 B	čitateľný výpis BTF: offsety polí v štruktúrach
<code>btbf.raw</code>	4 104 445 B	surový <code>/sys/kernel/btbf/vmlinux</code> z host'a, z ktorého <code>btbf.txt</code> vznikol
<code>boot.json</code>	398 B	doklad, z ktorého bootu profil je
<code>README.md</code>	6 039 B	ako a kedy adresár vznikol, akými príkazmi

`kallsyms.txt` — adresy symbolov. Symbol je pomenované miesto v jadre: `init_task` (štruktúra procesu s PID 0), `linux_banner` (reťazec s verziou jadra), `modules` (hlavička zoznamu modulov), `init_top_pgt` (vrchol tabuliek stránok), `socket_file_ops`.

Prečo 92 698 symbolov, keď súbor má 98 695 riadkov. Toto je presne otázka, ktorú komisia položí, keď si súbor otvorí. Formát riadku je `<adresa> <typ> <meno> [modul]` a v jadre je množstvo **statických** symbolov s rovnakým menom v rôznych prekladových jednotkách — napríklad `__func__.0` sa v tomto súbore vyskytuje 387-krát, `__key.0` 138-krát. Trieda `Profile` ukladá symboly do slovníka `self.sym` podľa mena a prvý výskyt vyhráva (`self.sym.setdefault(p[2], addr)` v `guestparse/profile.py`), takže duplicity zaniknú. Vlastné overenie dnes:

```
$ wc -l profiles/debian12-6.1.0-42-cloud-amd64/kallsyms.txt
98695
$ awk 'NF>=3 {print $3}' profiles/.../kallsyms.txt | sort -u | wc -l
92698
$ awk 'NF>=3 {print $3}' profiles/.../kallsyms.txt | sort | uniq -d | wc -l
1070
```

Teda 98 695 riadkov, z toho 92 698 rôznych mien; 1070 mien sa opakuje a 5 997 riadkov sú tieto opakovania. `boot.json` zapisuje počet **riadkov** (`kallsyms_lines: 98695`), nástroj hlási počet **symbolov** (92698). Obe čísla sú správne a merajú niečo iné.

`btbf.txt` — offsety polí v štruktúrach. BTF (BPF Type Format) je popis typov, ktorý si moderné jadro nesie v sebe. Z neho sa dozvieme tých 2976. Offsety sa **zámerne nečítajú z hlavičkových súborov jadra**: rozloženie štruktúry závisí od konfigurácie prekladu (`CONFIG_*`), nie iba od verzie jadra, takže číslo opísané zo zdrojákov by pri inom `.config` ticho ukazovalo na iné pole. Parser BTF výpisu je funkcia `btbf_offsets()` v `profile.py`; zostupuje aj do anonymných unionov (`_flatten()`), lebo práve v nich sú v jadrách 6.x uložené napríklad IP adresy v `sock_common`.

boot.json — **jediný priamy doklad o boote**. Obsahuje `boot_id`, čas štartu host'a, kedy sa profil odobral, verziu jadra, adresy `init_task` a `linux_banner` v čase odberu a `kallsyms_lines`. Z neho sa berie riadok, ktorý `guestparse` info vypisuje:

```
profil z bootu 82b778a3-89f8-4b04-b518-5db8eb4ab4, odobraty 2026-09-19T13:19:15+02:00
porovnaj s hostom: cat /proc/sys/kernel/random/boot_id
```

Bez `boot.json` by nástroj povedal „profil nemá `boot.json` — z ktorého štartu host'a je, zistiť neviem“ (metóda `Profile.describe()`). Na tomto súbore stojí diagnóza v časti 5.9.

Rozdiel medzi oboma hlavnými časťami je v tom, čo ich zneplatní:

časť profilu	viaže sa na	reštart host'a
<code>btf.txt</code> / <code>btf.raw</code> (offsety polí)	verziu a konfiguráciu jadra	prežije
<code>kallsyms.txt</code> (adresy)	konkrétny štart jadra	neprežije

Dôvodom je **KASLR** (Kernel Address Space Layout Randomization). Jadro sa pri každom štarte nahrá do pamäte na náhodne posunuté miesto. Robí sa to preto, aby útočník nevedel dopredu, na akej adrese je ktorá funkcia. Posun je pre celé jadro rovnaký — všetky symboly sa hýbu spolu.

Nameraný dôkaz z tohto projektu (`docs/LIMITACIE.md`, L17). Doména bola 18. 9. vypnutá a 19. 9. naštartovaná:

```
včerajší boot: ffffffff9711f560 D linux_banner ffffffff97a1aa40 D init_task
dnešný boot: ffffffff97b191f560 D linux_banner ffffffff97b221aa40 D init_task
```

Rozdiel je pri oboch symboloch rovnaký. Profil je teda pre `kallsyms` jeden **na boot**, nie jeden na verziu jadra.

5.3 Ako sa nájde posun: kotva, nie odhad

Ak je všetko posunuté o neznámu hodnotu, ako ju zistíme? Metóda je v `GuestView.resolve()` a stojí na dvoch krokoch.

Krok 1 — kandidáti. Celá snímka sa prehľadá na reťazec `"Linux version "`. Každý nájdený výskyt je kandidát na fyzickú adresu symbolu `linux_banner`. Z neho vyplýva posun:

```
shift = pa - (banner_va - 0xffffffff80000000)
```

Pomocná funkcia `_find()` spája susedné stránky, aby sa reťazec na hranici stránok nestratil. Čo je tá konštanta `0xffffffff80000000`, vysvetľuje časť 5.4 — je to `__START_KERNEL_map`.

Krok 2 — overenie. Kandidátov je viac než jeden: ten istý text je aj v logu jadra a v dátach `/proc`. Preto sa každý kandidát **overí**: z jeho posunu sa dopočíta adresa `init_task`, prečíta sa 16 bajtov na offsete `comm` a musí tam byť reťazec `"swapper/0"` — meno nečinného procesu s PID 0, ktorý v každom bežiacom Linuxe existuje.

Podstatné je, že overenie je **nezávislé od nálezu**. Banner nás priviedol k číslu; `swapper/0` je druhý, nesúvisiaci symbol, ktorý pri zlom posune sedieť nemôže. Posun sa teda nehádže — potvrdzuje sa.

Vlastný beh dnes nad snímkou z 19. 9.:

```
$ python3 -m guestparse info --snapshot data/raw/20260919T110155Z_once \
--profile profiles/debian12-6.1.0-42-cloud-amd64
snímka: vmicd (89483 stránok po 4096 B)
profil: 92698 symbolov, struktury: fdtable, file, files_struct, module, sock_common, socket, task_st
profil z bootu 82b778a3-89f8-4b04-b518-5db8eb4ab4, odobraty 2026-09-19T13:19:15+02:00
banner PA: 0x771f560 (kandidat 3)
posun jadra: -0x2a200000
page_offset: 0xfffff8de40000000
```

Platný bol až **tretí** kandidát. Vo staršej snímke z 18. 9. (`docs/ARCHITEKTURA.md`, časť 7.2) to bol **desiaty**. Bez overovania by sa vzal prvý a všetko ďalšie by bolo nezmysel.

5.4 Preklad adries: tri cesty

Jadro pracuje s **virtuálnymi adresami** (VA) — adresami vo svojej vlastnej mape. V snímke máme **fyzické adresy** (PA). Prevod medzi nimi má na x86_64 tri rôzne cesty a `GuestView.to_pa()` ich skúša v tomto poradí:

oblasť	vzťah	funkcia
obraz jadra	$PA = VA - 0xfffffffff800000000 + \text{posun}$	<code>ktext_pa()</code>
priamy mapping	$PA = VA - \text{page_offset_base}$	<code>direct_pa()</code>
moduly a <code>vmalloc(0xfffffffffc...)</code>	prechod tabuliek stránok	<code>walk_pa()</code>

Čo je `0xfffffffff800000000` a prečo sa odčítava. V jadre Linuxu sa táto konštanta volá `__START_KERNEL_map`. Je to virtuálna adresa, na ktorú sa pri štarte namapuje **obraz jadra** — jeho kód a statické dáta. Linker pri preklade jadra priradil každej funkcii a každej premennej virtuálnu adresu práve v tomto okne (preto všetky adresy v `callsyms` začínajú `0xfffffffff8...`). Vzťah medzi virtuálnou a fyzickou adresou je v tomto okne lineárny: prvý bajt obrazu jadra leží na nejakej fyzickej adrese a všetko ostatné za ním v rovnakom poradí. Keď teda od virtuálnej adresy odčítame začiatok okna, dostaneme **posun vnútri obrazu jadra**, a keď k nemu pripočítame, kde obraz vo fyzickej pamäti začína (to je náš posun z časti 5.3), dostaneme fyzickú adresu. Odčítanie `0xfffffffff800000000` teda nie je trik, je to prechod z „adresy v mape jadra“ na „poradové číslo bajtu v obraze jadra“.

Čo je priamy mapping (direct mapping) a prečo existuje. Jadro okrem svojho obrazu potrebuje pristupovať aj k *celej* fyzickej pamäti stroja — k stránkam procesov, k vyrovnávacím pamätiam, k dátam ovládačov. Aby to nemuselo pre každú stránku vyrábať dočasné mapovanie, namapuje si na x86_64 **celú fyzickú RAM naraz, lineárne, do jedného súvislého okna** virtuálneho priestoru. Fyzická adresa 0 je na začiatku okna, fyzická adresa 1 GiB o 1 GiB ďalej, a tak ďalej. Kde to okno začína, hovorí premenná jadra **`page_offset_base`** (je to premenná, nie konštanta, práve preto, že aj toto okno sa pri štarte randomizuje). Prevod je teda jedno odčítanie: $PA = VA - \text{page_offset_base}$. `resolve()` si `page_offset_base` prečíta hneď po nájdení posunu; v meranom behu bola `0xfffff8de400000000`.

Tretia cesta lineárna **nie je**. Moduly jadra sa nahrávajú za behu do oblasti od `0xfffffffffc00000000` a ich stránky ležia v RAM roztrúsene — Jadro im prideli, čo je práve voľné. Horná hranica lineárnej vetvy je preto v kóde konštanta `MODULES_VADDR`; nad ňou by odčítanie ticho vrátilo cudziu stránku.

Konkrétny prípad z **vlastného behu 2026-09-19** nad snímku `data/raw/20260919T110155Z_once` — modul `binfmt_misc` na $VA = 0xfffffffffc03de400$ (`GuestView.modules()`, `walk_pa_detail()` a `ktext_pa()` na tej istej adrese):

prechod tabuliek stránok (`to_pa`): $PA = 0x34d0400$ (správne, bez chyby)
lineárny výpočet (`ktext_pa`) by dal: $PA = 0x161de400$ (cudzia stránka)

Rozdiel je rádovo 300 MiB — lineárny výpočet by teda ticho prečítal úplne inú časť pamäte a vypísal by z nej „meno modulu“, ktoré by bolo smetou. Preto sa pre oblasť modulov ide tabuľkami stránok.

5.5 Čo sú tabuľky stránok

Pamäť je rozdelená na **stránky** po 4096 bajtoch (časť 2.4). Procesor prekladá virtuálnu adresu na fyzickú pomocou stromu tabuliek, ktorý má na x86_64 štyri úrovne: **PGD** → **PUD** → **PMD** → **PTE**. Každá tabuľka je jedna stránka s 512 položkami po 8 bajtoch.

Virtuálna adresa sa rozreže na kúsky po 9 bitov a každý kúsok je index do jednej úrovne. Pre `0xfffffffffc0560040`:

úroveň	bity	index
PGD	47:39	511
PUD	38:30	511
PMD	29:21	2
PTE	20:12	352

Spodných 12 bitov je offset v stránke (212 = 4096). Metóda `walk_pa_detail()` presne toto robí: začne v `init_top_pgt` (vrchol tabuliek jadra), na každej úrovni prečíta 8-bajtovú položku, skontroluje bit `PRESENT` (bit 0) a z položky vytiahne adresu ďalšej tabuľky.

Jedna odbočka: jadro vie namapovať aj **veľkú stránku** — 2 MiB namiesto 4 KiB (a na úrovni PUD dokonca 1 GiB). Vtedy sa strom nedokončí a preklad končí už na PMD, resp. PUD. Pozná sa to podľa bitu **PSE** (bit 7). `walk_pa_detail()` to kontroluje riadkom `if level in (1, 2) and (ent & self.PTE_PSE)`.

Keď prechod zlyhá, metóda nevráti len `None`, ale aj **dôvod**: `bez_korena` (nepoznáme posun), `chyba_tabulka` (tabuľku snímka neobsahuje) alebo `chyba_položka` (tabuľka v snímke je, ale položka v nej nie je prítomná — jadro tú adresu nemapuje). Na tomto rozlíšení stojí diagnóza v časti 5.9.

5.6 Krížová kontrola prekladu

Lineárny výpočet a prechod tabuliek sú dve nezávislé cesty k tej istej odpovedi. Ak posun sedí, **musia dať rovnaké číslo**. Ak by posun bol náhodný nález, lineárna vetva by vrátila hocičo a tabuľky niečo iné.

Robí to `GuestView.translation_check()` a vypisuje info. Vlastný beh dnes nad snímkom `data/raw/20260919T110155Z_once`

krizova kontrola prekladu (lineárne vs. tabulky stranok):

```
linux_banner    0x771f560      == 0x771f560      zhoda
init_task       0x801aa40      == 0x801aa40      zhoda
```

Beh nad snímkom z 18. 9. (`docs/ARCHITEKTURA.md`, časť 7.2) dal to isté s inými adresami: `linux_banner 0x16f1f560 == 0x16f1f560, init_task 0x1781aa40 == 0x1781aa40`.

5.7 Rekonštrukcia: prechod spájaných zoznamov

Teraz už vieme čítať. Zostáva vedieť, kam.

Jadro Linuxu spája objekty **spájaným zoznamom** (`struct list_head`): malá dvojica ukazovateľov `next/prev`, ktorá je **vnorená priamo v strede** väčšej štruktúry. Ukazovateľ v zozname teda neukazuje na začiatok `task_struct`, ale doprostred — na pole `tasks`. Preto sa offset **odčítava**:

```
tpa = self.to_pa(nxt - m["tasks"]) # view.py, processes()
```

V jazyku C to robí makro **container_of**. Je to štandardný idiom jadra Linuxu: „mám ukazovateľ na člen štruktúry, chcem ukazovateľ na celú štruktúru“. Robí sa to jediným odčítaním — od adresy člena sa odpočíta jeho offset vnútri typu. Tu je to odčítanie čísla **2192**. Na tomto jednom odčítaní stojí celá rekonštrukcia procesov: keby bol offset iný, program by čítal `comm` z cudzieho miesta a vypísal by mená, ktoré by vyzerali ako smeti — alebo, horšie, ako niečo zmysluplné.

Procesy (`processes()`): začne sa v `init_task.tasks`, ide sa po `next`, kým sa zoznam nevráti do hlavičky. Z každého uzla sa prečíta `comm` (16 B), `pid` (offset 2416), `tgid` (2420) a `mm` (2272) — proces s `mm == 0` je vlákno jadra. Strop je 8192 položiek, uzly sa zapamätávajú kvôli detekcii cyklu a PID musí byť v rozsahu do `PID_MAX`.

Moduly (`modules()`): hlavička `modules` je premenná v obraze jadra, ale samotné `struct module` ležia v oblasti modulov, takže každý článok potrebuje prechod tabuliek stránok. Meno je pole `name` na offsete 24 (overené cez `Profile.member('module', 'name')`).

Sokety (`sockets()`): pre každý používateľský proces sa prejde tabuľka deskriptorov (`task_struct.files` offset 3048 → `files_struct.fdt` offset 32 → `fdtable.fd` offset 8) a otvorený súbor sa označí za soket vtedy, keď sa jeho `file->f_op` (offset 40) rovná adrese symbolu `socket_file_ops`. To nie je heuristika — je to porovnanie s adresou z profilu host'a (dnes `0xffffffffb18f6920`). Odtiaľ vedie cesta cez `file->private_data` (offset 200) na `struct socket`, `socket.sk` (offset 24) a `sock_common` s adresami, portami a stavom. Protokol sa **neháda z čísla portu**: porovnáva sa ukazovateľ `skc_prot` so symbolmi `tcp_prot`, `udp_prot`, `raw_prot` a ich IPv6 dvojčikami.

Snímka je živá — VM sa počas čítania nezastavuje (L1), takže stránky sú z rôznych okamihov a zoznam sa môže roztrhnúť. Preto každý prechod vracia pole `truncated` a `stop_reason`. Polovičný zoznam sa nesmie dať zameniť za skutočný stav host'a.

Vlastné behy dnes nad snímkom `data/raw/20260919T110155Z_once`:

```
$ python3 -m guestparse ps      ... spolu: 78 procesov      (návratový kód 0)
$ python3 -m guestparse lsmod ... spolu: 47 modulov      (návratový kód 0)
$ python3 -m guestparse ss      ... spolu: 13 socketov     (návratový kód 0)
$ python3 -m guestparse checks
sys_call_table    451 položiek v [0xffffffffb0800000, 0xffffffffb1601ef2), nalezov: 0
procesy krizovo   tasks 78 vs. children/sibling 78, nalezov: 0
```

moduly krizovo modules 47 vs. module_kset 47 (z 105 kobjektov), nalezov: 0
nalezov spolu: 0 (z toho hookov v tabulke volani: 0)

K tejto konkrétnej snímke pozemná pravda uložená nie je, takže sú to výpisy, nie validácia.

5.8 Validácia proti pozemnej pravde

Rekonštrukcia môže byť sebavedomá a zároveň nesprávna. Overuje sa preto proti **pozemnej pravde** (ground truth) — tomu, čo o sebe hlási host' sám zvnútra.

Protokol (guestparse/validate.py): príkazy sa v host'ovi spustia **tesne pred snímkou a tesne po nej**. Porovnáva sa iba **stabilná množina** — PID prítomný v oboch behoch, teda proces, ktorý bežal cez celé okno odberu. Proces, ktorý sa objaví len v jednom behu ps, nie je ani chyba parsera, ani nález.

Dva rôzne výpisy soketov, ktoré sa nesmú miešať. scripts/root_run.sh odoberá oba a ukladá ich pod rôznymi menami (riadky 496–497 skriptu):

súbor	príkaz	čo obsahuje
ss_before.txt / ss_after.txt	ss -tuarp	všetky sokety TCP a UDP vrátane nadviazaných spojení
ss_listen_before.txt / ss_listen_after.txt	ss -tulpn	iba počúvajúce sokety

Keď sa rekonštrukcia porovnáva proti ss -tuarp, v pozemnej pravde je aj SSH relácia, cez ktorú sa odber robil, aj krátkodobý DNS dotaz — a tie sa medzi „pred“ a „po“ menia. Keď sa porovnáva proti ss -tulpn, pozemná pravda je iba zoznam počúvajúcich soketov a zo snímky sa navyše vidia nenaviazané a nadviazané sokety, ktoré v nej z definície nie sú. Obidva pohľady sú správne, len merajú niečo iné, a v tabuľke sa musia rozlíšiť. Artefakt data/results/2026-09-18_zmrazeny_host/summary.json to robí správne — má dva samostatné kľúče sokety_voci_ss_tulpn a sokety_voci_ss_tuarp.

Druhá jemnosť sú mená. Jadro drží v comm 16 bajtov, teda 15 znakov, ale ps tlačí pri vláknoch jadra dlhšie meno. Porovnanie preto pozná tri pravidlá — exact, truncated, worker — a **každé použitie počíta**, aby bolo v reporte vidno, koľko zhôd je doslovných.

Namerané výsledky (všetky hodnoty z uvedených artefaktov):

beh	pozemná pravda soketov	procesy (stabilné/nájdené)	moduly	sokety: sedí / chýba / navyše
2026-09-18, validate_20260918T154914Z_ssbefore.json	ss -tuarp	80 / 80 (100 %)	51 / 51	11 / 1 / 1
2026-09-18, validate_20260918T154914Z_ssafter.json	ss -tuarp	80 / 80 (100 %)	51 / 51	10 / 2 / 2
2026-09-18, zmrazený host', 2026-09-18_zmrazeny_host/validate2.json	ss -tulpn	81 / 81 (100 %)	51 / 51	10 / 0 / 2
2026-09-19, data/sessions/20260919T120148Z_validate/porovnanie.json	ss -tulpn	78 / 78 (100 %)	50 / 50	10 / 0 / 2

Vo všetkých riadkoch: chýbajúcich procesov 0, falošných 0, nezhoda mien 0, prechod zoznamov neprerušný (processes_truncated: false). Pravidlá zhody mien v poslednom behu: 63 doslovných, 12 worker, 3 orezané. Kontroly integrity prezreli 451 položiek sys_call_table s nula nálezmi.

Čo znamenajú chýbajúce a „navyšé“ sokety:

- **chýbajúce** (iba pri pozemnej pravde ss -tuarp): SSH relácia, cez ktorú sa pozemná pravda odoberala, a jeden DNS dotaz — oboje vzniklo alebo zaniklo počas odberu. V ssafter sú to doslova TCP 192.168.122.100:22 → 192.168.122.1:46882 a UDP 192.168.122.100:57006 → 192.168.122.1:53.

- **navyše:** nenaviazaný UDP soket `systemd-networkd (0.0.0.0:0)` a nadviazaný DNS dopyt (v behu z 19. 9. 192.168.122.100:55497 → 192.168.122.1:53). Pri pozemnej pravde `ss -tulpn` sú to práve tie sokety, ktoré počúvajúci výpis z definície neukazuje. Jeden soket protokolu RAW sa vykazuje samostatne v `snapshot_other_protocols`.

Popísané je to v `docs/kontroly.md`.

Jeden riadok bez porovnania. Artefakt `data/results/validate_20260918_vmic-val.json` sa v staršej verzii tohto dokumentu uvádzal ako „moduley neuoložené / sokety neuoložené“. Presne je to takto: v tom behu sa `validate` spustil bez prepínačov `--lsm` a `--ss`, takže artefakt má `modules.compared = false` a `sockets.compared = false` s dôvodom „bez `--lsm` nie je s čím porovnať“ — pričom zo samotnej snímky parser vyčítal **47 modulov a 13 soketov**. Chýba teda pozemná pravda, nie výsledok parsera.

5.9 Čo to nevie a čo znamenajú návratové kódy

Prechod sa nemusí uzavrieť. Živá snímka (L1). Vtedy je výsledok dolná hranica a je označený `NEUPLNE`.

Profil z iného bootu. Toto je najzákernejší prípad, lebo nástroj vyzerá, že funguje: posun sa **nájde** (banner je v pamäti bez ohľadu na to, z ktorého bootu je `kallsyms`), procesy sa vypíšu. Ale tabuľky stránok sú indexované skutočnými virtuálnymi adresami **tohto** štartu, takže stará VA v nich nie je. Krížová kontrola preto spadne. Zmerané 19. 9.: prechod skončil na úrovni PMD s kódom chyba_položka, položka `PMD[184]` pre `linux_banner` a `PMD[189]` pre `init_task`; výpisy boli označené `NEUPLNE`.

Od 19. 9. sa to nedá prehliadnuť: diagnózu `GuestView.profile_boot_mismatch()` volá `guestparse/cli.py` pred **každým** podpríkazom a pri nesúlade vypíše `NESULAD PROFILU` a skončí kódom 5 (`EXIT_PROFILE_MISMATCH`). `validate` sa pritom nespustí vôbec, aby po sebe nenechal výsledkový súbor z cudzieho profilu. Drží to `guestparse/tests/test_profile_boot.py`.

Návratové kódy `guestparse` (hlavička `guestparse/cli.py`, riadky 25–52). Komisii sa oplatí ich vedieť všetky, lebo práve nimi sa obhaja veta „nula nálezov je platný výsledok, nie ticho“:

kód	konštanta	znamená
0	<code>EXIT_OK</code>	všetko prebehlo a nič sa nenašlo
1	<code>EXIT_FINDINGS</code>	kontrola niečo našla. Prečo nie 0: príkaz sa dá zaradiť do skriptu a „našiel som hook“ sa nesmie stratiť v úspešnom kóde
2	<code>EXIT_ERROR</code>	chyba vstupu: snímka sa nedá otvoriť, profil sa nedá načítať, posun sa nenašiel
3	<code>EXIT_NOT_IMPLEMENTED</code>	podpríkaz nie je implementovaný (chýba modul balíka)
4	<code>EXIT_INCONCLUSIVE</code>	nič sa nenašlo, ale aspoň jedna kontrola sa neuzavrela. Nula z neuzavretej kontroly nie je dôkaz čistoty, preto sa nesmie hlásiť ako 0
5	<code>EXIT_PROFILE_MISMATCH</code>	profil je z iného štartu jadra než snímka

Poradie prebívania: keď sú nálezy aj neuzavreté kontroly naraz, vyhráva 1 (nález je silnejšia správa). Kód 5 prebíja aj 1, aj 4 — nález zo zlého profilu nálezom nie je, lebo nejde o „neviem“, ale o chybu vstupu.

Parsuje sa pamäť jadra, nie pamäť procesov (L4). Preklad pokrýva priestor jadra. Obsah pamäte používateľských procesov — argumenty, halda, načítané knižnice, spustiteľný kód procesu — sa neparsuje. Chýba na to vstup: preklad používateľských adries potrebuje hodnotu registra `CR3` konkrétneho vCPU alebo `mm_struct->pgd`, a backend registru vCPU nečíta. Z modulu `kvm` berie mapu memslotov a z vCPU iba ich počet (`atomic_t online_vcpus`, ktorý sa v `sidecar`i objaví ako `"num_vcpus": 2`). Overiteľné jedným príkazom: `grep -rniE "cr3|vcpu|\bregs\b|register" guestparse/*.py | wc -l` vráti nulu.

6. Príznyaky a model: od snímky k číslam a k TCN

6.1 Prečo vôbec príznyaky

Jedna snímka pamäte virtuálneho stroja hyptcn-guest je 2,02 GiB dát — presne 528 417 stránok po 4096 bajtoch. Neurónová sieť s takým vstupom pracovať nevie: mala by miliardy vstupov a desiatky sedení dát. Preto sa medzi snímku a model vkladá krok, ktorý z tých 2 GiB urobí niekoľko desiatok čísel. Tie čísla sa volajú **príznyaky** (features).

Príznyak je merateľná vlastnosť, ktorú si vyberá človek, nie sieť. Je to kompromis: čo do príznyaku nedáte, to model nikdy neuvidí. Zato to, čo tam dáte, viete pomenovať a obhájiť — a to je pri diplomovej práci dôležitejšie než pár percent presnosti.

Cesta má dve poschodia. Najprv **per-bin vektor** (pamäť po kúskoch), potom **snímkový vektor** (22 čísel na snímku). Prvé poschodie počíta modul zberača v C (vmicollect/src/perbin.c) a zapisuje ho do JSON sidecaru vedľa snímky; nezávislá Pythonová referencia features/perbin.py ho počíta znova a porovnáva. Druhé poschodie je čisto Python: features/snapshot.py.

6.2 Per-bin vektor: pamäť rozdelená na biny

Bin je súvislý kus fyzickej pamäte pevnej veľkosti. Východzia veľkosť je 16 MiB (vmicollect/src/config.c, cfg->feat_bin_bytes = 16u << 20). Index binu je $GPA \gg \log_2(\text{bin_bytes})$, teda je viazaný na fyzickú adresu, nie na poradie memslotu. Veľkosť musí byť mocnina dvojky, inak taký posun neexistuje.

Kľúčové rozhodnutie: **binuje sa iba nad memslotmi**. Memslot je oblasť, ktorú hypervízor virtuálnemu stroju skutočne dal ako RAM. Fyzický adresný priestor x86 je deravý (časť 2.6). Doména hyptcn-guest má 2,02 GiB RAM rozprestretú v 4 GiB priestore, v desiatich memslotoch.

spôsob	počet binov
naivne cez max_paddr (4 GiB / 16 MiB)	256, z toho 125 trvale prázdnych
naivne cez veľkosť RAM (2,02 GiB / 16 MiB)	asi 129
skutočnosť (iba nad memslotmi)	131

Preto nie 256: 125 binov by bolo navždy nulových a model by sa učil na výplň. Skutočných 131 sú biny 0–127 (RAM od 0x0 po 2 GiB, s dierou), plus bin 251 (0xf0000000), bin 254 (0xf0000000, jediná stránka) a bin 255 (0xffffc0000). Bin 128 neexistuje, lebo RAM končí presne na 2 GiB. Bin 0 má 4064 stránok, nie 4096 — chyba mu VGA diera 0xA0000–0xBFFFF (vlastný výpočet: 160 stránok prvej oblasti + 3904 stránok druhej oblasti po hranicu 16 MiB). Bin, ktorý nepretína žiadny memslot, nevzniká vôbec; nevypĺňa sa nulami.

Za každý bin sa počítajú tieto čísla (features/PERBIN.md):

- **pages_total** — koľko stránok binu je naozaj podložených memslotmi. Menovateľ všetkých pomerov.
- **pages_changed** — koľko stránok sa od predchádzajúcej snímky zmenilo. „Zmenená“ znamená, že delta writer ju zapísal do .vmicd súboru, lebo jej 64-bitový hash sa líšil od predchádzajúceho cyklu.
- **changed_ratio** = pages_changed / pages_total. Podiel, nie počet — aby sa biny rôznej veľkosti dali porovnať.
- **zero_ratio** — podiel úplne nulových stránok. Nulová stránka je pamäť, ktorú host prideliť a ešte nepoužil. Klesajúci zero_ratio znamená, že niekto berie pamäť.
- **entropy_mean** — priemerná Shannonova entropia zmenených stránok.
- **has_changed** — 0/1.

Shannonova entropia (prvý z dvoch významov slova „entropia“ v tomto dokumente). Je to miera nepredvídateľnosti **obsahu stránky**. Stránka sa rozloží na histogram 256 hodnôt bajtov a spočíta sa $H = -\sum p \cdot \log_2(p)$, výsledok je v bitoch na bajt, od 0 po 8. Stránka vyplnená jedným bajtom má 0. Bežný text alebo strojový kód má rádovo 3–6. Šifrované alebo komprimované dáta majú takmer 8, lebo bajty v nich vyzerajú náhodne. Preto sa entropia meria: ransomvér, ktorý prepisuje súbory šifrovaným obsahom, by mal entropiu zmenených stránok zdvihnúť. Že to tak naozaj je, táto práca nemeria (časť 6.8).

Prečo je has_changed samostatný príznyak. Keď sa v bine nič nezmenilo, entropy_mean vyjde 0 — ale nie preto, že by pamäť bola prázdna, ale preto, že nebolo z čoho priemerovať. Bez has_changed by model tieto dva prípady nerozoznal. Ticho doplnená nula je v tomto repozitári zakázané pravidlo, nie štýl.

C a Python počítajú entropiu inými cestami (C tabuľkovou formou $H = \log_2(n) - (1/n) \cdot \sum c_j \cdot \log_2(c_j)$, referencia priamo). Krížová kontrola z 2026-09-18 porovnala 6 snímok $\times$ 131 binov $\times$ 8 polí = **6288 polí, 0 rozdielov**;

tolerancia bola $5 \cdot 10^{-7}$, najväčší skutočný rozdiel `changed_ratio` $4,84 \cdot 10^{-7}$ a `entropy_mean` $4,55 \cdot 10^{-7}$, a počet polí mimo tlačenej presnosti (šesť desatinných miest, ktoré C vypisuje) bol 0 (`data/results/perbin_crosscheck_20260918.json` kľúče `porovnanie.rozdielov`, `porovnanie.max_abs_rozdiel`, `porovnanie.mimo_tlacenej_presnosti`). Výpočet stojí 51,5–52,4 ms na delta cyklus (medián, `latency_vector_20260918.json`) plus 3,498 μ s na zmenenú stránku za entropiu (`perbin_c_20260918/perbin_c_20260918.json`, $n = 6$).

6.3 Spojený vektor: 22 čísel na snímku

Per-bin vektor má 131 riadkov, a ten počet závisí od veľkosti pamäte konkrétnej VM. Model, ktorý by bral biny ako pevný počet vstupov, by fungoval len na VM rovnakej veľkosti. Preto `features/snapshot.py` cez biny **agreguje** a vyrobí jeden vektor pevnej dĺžky. Cenou je, že sa stráca informácia o tom, ktorá oblasť pamäte sa menila; per-bin vektor tým nezaniká, zostáva v `sidecari`.

Kontrakt je konštanta `MENA` vo `features/snapshot.py` a má 22 položiek (overené vlastným behom `python3 -c "from features.snapshot import MENA; print(len(MENA))" → 22`):

#	meno	čo meria
1	<code>mem_changed_ratio</code>	$\Sigma \text{ pages_changed} / \Sigma \text{ pages_total}$ — koľko percent pamäte sa zmenilo
2	<code>mem_bins_active</code>	podiel binov s <code>has_changed = 1</code> — v koľkých miestach pamäte sa robí
3	<code>mem_changed_max</code>	najvyšší <code>changed_ratio</code> cez biny — najhorúcejšie miesto
4	<code>mem_changed_p95</code>	95. percentil <code>changed_ratio</code> — ako vyzerá horúca menšina binov
5	<code>mem_changed_spread</code>	normalizovaná entropia rozdelenia zmien medzi biny : 0 = všetko v jednom bine, 1 = rovnomerne
6	<code>mem_entropy_mean</code>	priemer <code>entropy_mean</code> , vážený počtom zmenených stránok
7	<code>mem_entropy_max</code>	najvyššia entropia cez biny so zmenou
8	<code>mem_zero_ratio</code>	podiel nulových stránok
9	<code>proc_total</code>	počet procesov host'a
10	<code>proc_user</code>	z toho používateľských
11	<code>proc_kernel</code>	z toho vlákien jadra
12	<code>proc_new</code>	PID, ktoré v predchádzajúcej snímke neboli
13	<code>proc_gone</code>	PID, ktoré zmizli
14	<code>mod_total</code>	počet načítaných modulov jadra
15	<code>mod_delta</code>	zmena počtu modulov oproti predchádzajúcej snímke
16	<code>sock_total</code>	počet soketov
17	<code>sock_listen</code>	z toho počúvajúcich
18	<code>sock_estab</code>	z toho nadviazaných spojení
19	<code>chk_syscall_hooks</code>	počet položiek <code>sys_call_table</code> , ktoré ukazujú mimo jadra
20	<code>chk_crossview</code>	počet krížových nezrovnalostí (objekt vidí jeden pohľad a druhý nie)
21	<code>ma_predchodcu</code>	0/1 — bola pred touto snímku iná?
22	<code>je_plna</code>	0/1 — je to plná snímka, alebo delta?

Dve rôzne veličiny s rovnakým menom „entropia“. Toto treba mať pripravené, lebo to je lacná otázka na obhajobe. Príznak 6 a 7 (`mem_entropy_mean`, `mem_entropy_max`) sú Shannonova entropia **obsahu stránok** — koľko bitov na bajt, od 0 po 8 (časť 6.2). Príznak 5 (`mem_changed_spread`) je entropia **rozdelenia počtu zmenených stránok**

medzi biny — teda nie „ako náhodne vyzerajú bajty“, ale „ako rovnomerne sú zmeny rozhádzané po pamäti“. Normalizuje sa na rozsah 0 až 1. Obe používajú ten istý vzorec $-\sum p \cdot \log_2(p)$, ale nad úplne inou premennou a v iných jednotkách. Miešať ich nemožno.

Príznamy 1–8 sa berú **výlučne** z bloku `features` v `sidecari`, ktorý počíta C modul. Keď tam nie je, funkcia skončí chybou. Dôvod: v jednom datasete nesmú byť čísla z dvoch implementácií, lebo by sa spätne nedalo zistiť, ktorý riadok je z ktorej. (Pomery sa pritom počítajú z celých čísel `pages_changed` a `pages_total`, nie z poľa `changed_ratio` — to C tlačí zaokrúhlené na šesť desatinných miest.)

Prečo dva indikátory navyše. `proc_new`, `proc_gone` a `mod_delta` potrebujú predchádzajúcu snímku. Pri prvej snímke reťazca žiadna nie je, takže sú nula — ale tá nula nie je meranie. `ma_predchodcu` = 0 to hovorí. Podobne v **plnej** snímke `delta writer` zapíše všetky nenulové stránky, takže „zmenené“ tam znamená „nenulové“ a príznamy 1–7 opisujú celý obsah pamäte, nie jej zmenu. Vlastný výpočet nad prvým `sidecarom` behu `data/raw/20260919T110017Z_run` (131 binov) to ukazuje: `mem_changed_ratio` = 0,169461 a `mem_zero_ratio` = 0,830539, súčet presne 1,000000; `output.changed_ratio` v tom istom `sidecari` je tiež 0,169461. To isté číslo s iným fyzikálnym významom. Preto je `plna`. Ten istý jav je priznaný ako obmedzenie L13.

Ticho doplniť nulu sa nesmie preto, že taký vektor **vyzerá správne a nie je**. Chyba by sa neprejavila pádom, ale horším modelom, a nikto by nevedel prečo. Alternatíva NaN sa nepoužíva, lebo by sa cez normalizáciu a okná rozliezla do všetkého.

6.4 Okná: model potrebuje postupnosť

Jedna snímka je okamih. Správanie je zmena v čase, takže vstup modelu nie je vektor, ale **okno** — pevný počet po sebe idúcich snímok. Tvar dát je `X[okno, čas, príznak]`, teda (N, L, F).

Východzia dĺžka je `DLZKA_OKNA` = 16 snímok (`features/windows.py`). Pri perióde zberu 2 s je to 32 s, pri 5 s 80 s — dosť na to, aby bola v okne vidieť fáza správania, a nie toľko, aby jedno okno pokrylo celý beh. Krok posunu je 1, takže sa okná prekrývajú.

Čo je „sedenie“ (session). Jeden súvislý beh zberu nad jednou doménou: jeden adresár so snímkami, jedna perióda, jeden priebeh záťaže v host'ovi. Sedenie je najmenšia jednotka, ktorá sa pri delení dát nesmie rozrezať.

Okno **neprekročí hranicu sedenia**. Nie je to kozmetika: susedné okná sa pri kroku 1 prekrývajú v 15 zo 16 snímok, takže testovacie okno by malo v tréningu takmer identického suseda. Okno sa ďalej neskladá cez diery v `seq` a cez príliš dlhú časovú medzeru.

Okno, ktoré by obsahovalo riadok s `je_plna` = 1 alebo `ma_predchodcu` = 0, nevznikne vôbec. Riadok sa z relácie nevyhadzuje — vyhodiť ho a susedov spojiť by znamenalo okno, v ktorom medzi dvoma riadkami chýba časový krok.

Prečo padli čísla predchádzajúcej iterácie (hypTcn002). Túto vetu treba mať presne, lebo dokument si ju inak sám seba protirečí. Podľa `HONESTY.md`, kapitola 2, je **spoločným dôvodom neplatnosti confound zo session-level labelov**: každé okno v rámci jednej session dostalo label podľa toho, čo sa v tej session spúšťalo, a benígne aj „malvérové“ sessions sa líšili úrovňou aktivity systému. Model sa teda naučil rozlíšiť „vyťažný stroj“ od „nečinného stroja“, nie malvér od benígneho softvéru. Miešanie okien z jednej session medzi tréningovú a testovaciu množinu je uvedené ako **druhý, doplnkový** dôvod — zosilnilo to, ale samo o sebe to nebola hlavná príčina. Doklad, že to tak je: keď sa split opravil, metriky klesli (0,86 namiesto 0,997), ale **confound zostal**, takže neplatné zostali aj tie nižšie čísla.

Z toho plynie pravidlo, ktoré drží `features/windows.py` aj `tcn/train.py`: okno nikdy neprekročí hranicu sedenia, sedenie sa nikdy nerozdelí medzi tréning a test, a label nesmie byť vlastnosť sedenia, ktorá koreluje s triedou.

6.5 Normalizácia, fit a transform, a čo je data leakage

Príznamy majú rôzne rozsahy: `proc_total` je okolo 78, `mem_changed_ratio` je 0,0007. Sieť by sa učila hlavne z veľkých čísel. **Z-score** to zrovná: od každého príznamu sa odčíta jeho priemer `mu` a vydelenie sa jeho smerodajnou odchýlkou `sd`, takže po transformácii má každý príznam priemer 0 a odchýlku 1.

Dva pojmy, ktoré sa v strojovom učení používajú neustále a tu sú prvýkrát:

- **fit** — „nauč sa parametre z dát“. Pri z-score to znamená: prejdí dáta a spočítaj `mu` a `sd` pre každý príznam.

- **transform** — „použi naučené parametre na dáta“. Teda: od každej hodnoty odčítaj mu a vydeľ `sd`. `transform` bez predchádzajúceho `fit` vyhodí v tomto kóde výnimku.
- **manifest** — súbor, do ktorého sa naučené parametre uložia spolu s poradím príznakov a jeho hashom. Pri načítaní sa poradie overuje, takže sa nedá použiť `fit` z inej verzie kontraktu.

Otázka je, **z čoho** sa mu `sd` počítajú. `features/normalize.py` má pravidlo: fitujú sa **iba na benígnych tréningových sedeniach**, nikdy na testovacích. Keby sa počítali zo všetkých dát, model by cez mu a `sd` videl niečo z testovacej množiny ešte pred vyhodnotením. To je **data leakage** — únik informácie z testu do tréningu. Výsledok potom vyzerá lepšie, než aký v prevádzke bude, a chyba sa nedá spätne nájsť v žiadnom jednom riadku kódu.

Fit sa robí **nad snímkami, nie nad oknami** — pri kroku 1 je tá istá snímka v 16 oknách a snímky zo stredu sedenia by dostali násobnú váhu. `ma_predchodcu` a `je_plna` sa nenormalizujú: sú to indikátory, ktorých jedinou úlohou je povedať, čo ten riadok je, a `z-score` by ten význam zmazal.

Čo je „split“. Rozdelenie dát na tréningovú a testovaciu množinu. Tu je **chronologický a session-disjunktný**: sedenie sa nikdy nerozdelí a staršie sedenia idú do tréningu, novšie do testu. Chronologický preto, že detektor sa v prevádzke učí z minulosti a rozhoduje o budúcnosti; opačné poradie by bolo predpovedanie minulosti zo znalosti budúcnosti.

6.6 TCN

Konvolúcia na časovom rade je kľzavý filter. Máte malé okno váh (jadro, kernel) dĺžky k a posúvate ho po časovej osi; v každej polohe spočítate vážený súčet a ten je výstup. Pri $k = 3$ sa každý výstupný krok počíta z troch susedných vstupných krokov. Rovnaké váhy platia všade po celej dĺžke — to je celý trik: vzorec „rýchly nárast zmien“ sa naučí raz a rozpozná sa kdekoľvek.

Kanály. Jedna konvolúcia nemá jeden filter, ale niekoľko — každý so svojimi váhami. Každý filter vyrobí vlastnú výstupnú postupnosť a tým postupnostiam sa hovorí **kanály**. Šesťnásť kanálov teda znamená šesťnásť rôznych detektorov vzorcov bežiacich súčasne nad tým istým vstupom. V tomto modeli je `kanaly = 16` (východzia hodnota v `tcn/model.py`). Vstup má 22 kanálov (jeden na príznak), prvý blok ich prevedie na 16 a ďalšie bloky už pracujú $16 \rightarrow 16$.

Kauzálna konvolúcia je taká, kde výstup v čase t závisí výlučne od vstupov v čase $\leq t$. Dosahuje sa to tým, že sa vstup dopĺňa nulami **iba zľava**, o $(k-1)$ -dilatácia vzoriek. V `tcn/model.py` je to jediný riadok: `F.pad(x, (self.pad, 0))`. Pri detekcii to treba preto, že detektor beží v čase — rozhodnutie o aktuálnej snímke sa nesmie opierať o snímky, ktoré ešte nenastali. Nekauzálny model by v offline vyhodnotení vyzeral lepšie a v prevádzke by sa nedal použiť. Nie je to iba tvrdenie v komentári: `tcn/tests/test_tcn.py::test_kauzalita` zmení vstup v čase $t+1$ a overí, že výstup v čase t sa nezmenil.

Dilatácia je medzera medzi vzorkami, ktoré jadro berie. Pri dilatácii 1 berie kroky $t, t-1, t-2$. Pri dilatácii 2 berie $t, t-2, t-4$. Pri 4 berie $t, t-4, t-8$. Bloky idú s dilatáciami 1, 2, 4, ... $2^{(B-1)}$, takže dosah rastie exponenciálne, kým počet vrstiev rastie lineárne. Bez dilatácie by sa na dosah 16 krokov pri $k = 3$ potrebovalo osem vrstiev; s dilatáciou stačia tri bloky.

Reziduálny blok. „Reziduum“ je zvyšok, rozdiel. Reziduálny blok je konštrukcia, kde sa výstup vrstiev **pripočíta k vstupu** namiesto toho, aby ho nahradil: $\text{výstup} = f(x) + x$. Vrstva sa teda neučí celú odpoveď, ale iba to, čo treba k vstupu pridať. Dve veci z toho plynú. Po prvé, model sa vie „naučiť nič nerobiť“ jednoducho tým, že $f(x)$ stlačí k nule, takže pridanie ďalšieho bloku nemôže model pokaziť. Po druhé, pri tréningu má gradient skratku priamo cez to sčítanie, takže sa neztráca v hĺbke siete. V kóde je to trieda `Blok` v `tcn/model.py`, konkrétne riadky:

```
s = x if self.skratka is None else self.skratka(x)
return torch.relu(y + s)
```

Keď sa počet kanálov medzi vstupom a výstupom líši (to je prípad prvého bloku: $22 \rightarrow 16$), skratka musí ten počet zosúladiť, a robí to konvolúcia s jadrom dĺžky 1 (`nn.Conv1d(vstup, kanaly, 1)`).

Dropout. Pri tréningu sa v každom kroku náhodne vynuluje istý podiel hodnôt (tu 0,1, teda 10 %). Sieť sa tým núti nespoliehať sa na jednu konkrétnu cestu — keď sa ktorákoľvek jednotka môže kedykoľvek stratiť, informácia musí byť zastúpená viackrát. Je to zábrana proti preučeniu (overfitting), teda proti tomu, aby sa model naučil naspamäť tréningové dáta namiesto pravidla. Pri vyhodnotení sa dropout vypína.

Gradient. Keď sa sieť učí, potrebuje vedieť, ako veľmi by sa výstup zmenil pri malej zmene každej váhy. Vektor týchto citlivostí sa volá **gradient** a počíta sa spätným chodom cez sieť. Používa sa aj mimo tréningu: test `test_recepcne_pole` meria dosah modelu tak, že sa pozrie, **na ktoré vstupné kroky je výstup vôbec citlivý**

— teda kde je gradient nenulový. To je pocktivejšie než dosadiť do vzorca, lebo vzorec by nezachytil chybu v implementácii doplnenia nulami.

Receptívne pole (RF) je počet vstupných krokov, ktoré ovplyvnia jeden výstupný krok. Každá konvolúcia ho predĺži o $(k-1) \cdot \text{dilatácia}$, blok má dve konvolúcie, takže

$$\text{RF} = 1 + 2 \cdot (k-1) \cdot (2^B - 1)$$

Pri východných $k = 3$ a $B = 3$ je $\text{RF} = 1 + 2 \cdot 2 \cdot 7 = 29$ snímok. Okno má 16, takže posledný časový krok vidí celé okno. Pri $B = 2$ by bolo $\text{RF} = 13 < 16$ a časť okna by do výstupu nevstúpila vôbec — to je chyba návrhu, nie vlastnosť, a funkcia `skontroluj_rf()` v takom prípade vyhodí výnimku namiesto toho, aby model ticho natrénovala. Vlastné overenie dnes: `recepne_pole(3, 3) → 29`.

Logity a softmax. Posledná vrstva modelu (`self.hlava = nn.Linear(kanály, triedy)`) dá na výstupe jedno číslo na triedu. Tie čísla sa volajú **logity**. Nie sú to pravdepodobnosti: môžu byť záporné, nemusia dávať súčet 1, a ich absolútna hodnota nemá jednotku. **Softmax** je funkcia, ktorá ich na pravdepodobnosti prevedie: každý logit sa umocní na e , a potom sa vydolí súčtom všetkých — výsledkom sú kladné čísla so súčtom presne 1. Preto výstup skórovania vyzerá ako dvojica, napríklad $[0.8521, 0.1479]$ (vlastný beh, časť 8.7). Keďže model **nie je natréňovaný**, tieto dve čísla nehovoria o stave VM nič; hovoria len to, že softmax dostal nejaké logity a spočítal z nich rozdelenie.

Prečo TCN a nie LSTM/GRU. Rekurentná sieť spracúva okno krok po kroku: krok t sa nedá počítať skôr, než doráta $t-1$. Konvolúcia počíta všetky časové kroky naraz, takže sa dá paralelizovať. Pri tréningu je aj cesta gradientu iná: v TCN vedie cez pevný počet vrstiev (tu tri bloky plus hlava), v rekurentnej sieti cez toľko krokov, koľko má sekvencia. Krátka a pevná cesta sa trénuje stabilnejšie. K tomu je dosah TCN **vypočítateľné číslo**, ktoré sa dá skontrolovať pred tréningom; u rekurentnej siete sa dosah len odhaduje.

Počet parametrov. Vlastné overenie dnes (`python3 -c "from tcn.model import TCN, pocet_parametrov; print(pocet_parametrov(TCN(22,2)))"`): pri $F = 22$ príznakoch, 2 triedach, 16 kanáloch, $k = 3$ a $B = 3$ má model **5394 parametrov**. Rádovo tisíce, nie milióny — zámerne, lebo korpus je malý. (Hlavička `tcn/model.py` ešte uvádza 5330 a 21 príznakov; to je stav pred pridania `je_plna` a je to zastaraný text, nie iné číslo — podľa `HONESTY.md` P12 platí kód a opravuje sa komentár.)

Slovo „port“ v poslednej vete o architektúre neznamená sieťový port. Architektúra je port — teda prenos, prepis do iného prostredia — bežného TCN podľa práce Bai, Kolter, Koltun 2018 („An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling“). Predloha je uvedená v hlavičke `tcn/model.py`; samotná implementácia je napísaná pre túto prácu.

6.7 Baseliney a prečo sú povinné

Baseline je jednoduchý model, proti ktorému sa ten zložitý meria. Bez neho sa nedá povedať, či zložitost' niečo priniesla. Všetky tri bežia nad **tými istými** oknami a tým istým splitom (`tcn/baselines.py`).

1. **logreg_priemer** — logistická regresia nad oknom spriemerovaným cez čas. Ukazuje, koľko sa dá dosiahnuť bez modelu sekvencií vôbec.
2. **bag_of_frames** — priemer a smerodajná odchýlka cez čas. Tento model **nevidí poradie snímok**: keď okno v čase zamiešate, jeho vstup sa nezmení (overuje `test_bag_of_frames_nevidi_poradie`). Preto je najdôležitejší. Celá práca stojí na tvrdení, že v dátach je časová informácia. Keď `bag_of_frames` dorovná TCN, znamená to, že poradie snímok nepomáha a časový model je zbytočný. Taký výsledok sa podľa `HONESTY.md` P8 reportuje v tej istej tabuľke, nie v prílohe.
3. **gru** — rekurentná sieť s porovnateľným počtom parametrov (zadanie žiada porovnanie s LSTM/GRU). „Porovnateľný“ nie je odhad: skrytý rozmer vyberá `skryte_pre_parametre()` tak, aby sa počty líšili čo najmenej. Vlastné overenie dnes (`skryte_pre_parametre(22, 2, 5394)`) dáva skrytý rozmer **32** a **5442 parametrov** oproti 5394 v TCN, teda rozdiel pod 1 %.

6.8 Čo sa netvrdí

Model nie je natréňovaný. Korpus neexistuje. Žiadna presnosť detekcie sa v tejto práci netvrdí.

Dôvod je vstup, nie kód. Na stroji, na ktorom práca vznikla, nie sú žiadne reálne malvérové vzorky. Bez nich niet čo označiť a niet na čom trénovať (`docs/LIMITACIE.md`, L16).

Šesť syntetických scenárov správania (`cpu_burn`, `mass_file_rewrite`, `proc_scan`, `anon_exec`, `fork_storm`, `idle`) bolo napísaných a odskúšaných, ale na tréning sa **vedome** nepoužili. Model natréňovaný na nich by nebol

detektorom malvéru, ale klasifikátorom šiestich skriptov, ktoré napísal autor práce — rozlišoval by presne tie vzorce, ktoré doňho autor vložil. Číslo z takého vyhodnotenia by v texte znelo ako presnosť detekcie, hoci by meralo zhodu s vlastným generátorom. Je to tá istá chyba, pre ktorú sa retrahovala predchádzajúca iterácia (časť 6.4 a kapitola 10).

A ešte jedna vec, ktorú treba povedať skôr, než sa komisia spýta: vetva zberu korpusu (scenarios/, scripts/collect_corpus.sh) bola z pracovného stromu **odstránená** (L16, commit da8ad2e; overené dnes — `ls scenarios` aj `ls scripts/collect_corpus.sh` vracajú „No such file or directory“). Dve označené sedenia v `data/raw/` (20260919T011728Z_mass_file_rewrite a 20260919T011815Z_fork_storm) majú `labels.json` so sha256 scenára, ktorý ich vyrobil, ale samotný scenár v strome nie je — takže tie dve sedenia sa dnes **nedajú reprodukovať** a nič ich nespracúva. Podrobne v časti 9.3.

Čo teda overené je: **mechanika celej cesty**. Beh `python3 -m tcn.train --syn` overuje, že okná neprekročia hranice sedenia, že split nemieša sedenia, že beh je deterministický a že sa sieť na triviálnej úlohe naučí. Metriky z neho sú 1,0, lebo generátor vyrába oddeliteľné triedy; uložené sú v `data/results/smoke/s synteticke_data: true` a `guest: null` presne preto, aby ich nikto nepomýlil s výsledkom. Do textu práce nepatria.

Vlastné behy dnes: `python3 -m pytest -q features/tests tcn/tests` prešiel **112 testov, 2 preskočené** (preskočené potrebujú snímky mimo repozitára), celý `python3 -m pytest -q` prešiel **192 testov a 21 preskočil**, pričom nástroj sám vypisuje upozornenie „preskocilo sa 21 testov - preskoceny test NIE JE presiel“.

Aj `tcn/score.py`, ktorý spája bežiaci zber s modelom, má náhodne inicializované váhy s pevným seedom 0. Číslo, ktoré vypíše, je dôkaz, že cesta *pamäť VM → snímka → vektor → okno → TCN → číslo* beží ako jeden celok — nie výrok o tom, či sa vo VM niečo deje. Upozornenie je zámerne na troch miestach: v docstringu, na začiatku každého výpisu a v každom JSON výstupe pod kľúčom `model_nat` renovany.

Pred komisiou to znie takto: cesta dát je zostavená a overená, bod zadania o vyhodnotení presnosti detekcie je nesplnený, a dôvodom je nedostupnosť vzoriek, nie nedostatok implementácie.

Poznámka k pôvodu kódu: moduly `features/` a `tcn/` vznikli pre túto prácu, s výnimkou architektúry TCN, ktorej predloha je uvedená v hlavičke `tcn/model.py`. Pôvod `vmcollect` vrátane `perbin.c` rieši časť 3.9 a je to jedno a to isté znenie v celom dokumente.

7. Ako sa to preloží a spustí

Táto kapitola je tu preto, aby sa dala pripraviť živá ukážka na obhajobe. Východiskom je `docs/PRIRUCKA.md` kapitoly 2 až 4, ale nie ako odpis: pri každom kroku je napísané, **čo** sa deje a **prečo**, a čo je najčastejšia príčina, prečo to nepôjde.

7.1 Čo treba mať

požiadavka	prečo
hostiteľ s KVM, QEMU a libvirt, na ňom doména so sledovaným systémom	bez <code>struct kvm</code> v jadre hostiteľa nie je čo čítať
<code>clang</code> s podporou cieľa BPF	BPF program sa prekladá do bajtkódu BPF, nie do x86; bežný <code>gcc</code> to nevie
<code>libbpf</code> , <code>zlib</code> , <code>libdl</code> , GNU make, Python 3	<code>libbpf</code> načítava program do jadra a rieši BTF; <code>zlib</code> je pre voliteľnú kompresiu
jadro hostiteľa preložené s CONFIG_DEBUG_INFO_BTF	odtiaľ sa berú offsety polí <code>struct kvm</code> ; bez BTF by museli byť zadrátované v kóde a pri inom jadre by ticho ukazovali inam
root na hostiteľovi , konkrétne <code>CAP_BPF</code> a <code>CAP_PERFMON</code>	načítanie BPF programu a čítanie cudzieho adresného priestoru
profil jadra host'a v <code>profiles/</code>	bez neho parser neurobí ani krok (časť 5.2)

Vnútri host'a netreba inštalovať nič. `qemu-guest-agent` alebo `ssh` potrebuje iba odber profilu a pozemnej pravdy, teda príprava a overovanie — nie samotný zber. Toto je dôležité rozlíšenie pri otázke o neinvazívnosti (`HONESTY.md` P6).

Prostredie, na ktorom všetky merania bežali, je v tabuľke v časti 8.2.

7.2 Preklad a testy — bez VM a bez roota

```
make -C vmicollect          # -> ./vmicollect/build/vmicollect
make -C vmicollect test     # plná snímka, tri delty, obnova reťazca, kontrolný súčet
python3 -m pytest -q       # testy parsera a príznakov
```

make test prejde celú cestu zberu nad syntetickým obrazom veľkosti 16 MiB, ktorý si sám vyrobí, a použije na to backend file (časť 4.3). Nepotrebuje teda ani root, ani doménu. Súčasťou sú dva testy v C: hole_test (kontrakt čítania cez dieru, časť 2.6) a perbin_test (per-bin vektor).

Vlastné behy dnes: make -C vmicollect test skončil selftest: VSETKO PRESLO, python3 -m pytest -q dal **192 prešlo a 21 preskočilo** (preskočené chcú validačnú snímku v /var/tmp/vmic-val alebo reťazec v /var/tmp/vmic-delta; nástroj to sám vypíše a pripomenie, že preskočený test nie je prešiel). make -C vmicollect hooks navyše zostaví build/example_hook.so pre ukážku hook API.

7.3 Profil jadra host'a

```
scripts/get_profile.sh -f root@192.168.122.100
```

Skript sa cez ssh spýta host'a na /proc/kallsyms a /sys/kernel/btf/vmlinux, uloží ich do profiles/<distro-jadro>/ a na hostiteľovi z btf.raw urobí čitateľný výpis btf.txt cez bpftool btf dump file btf.raw format raw. Výpis BTF sa robí na hostiteľovi zámerne, aby sa kvôli profilu do host'a nič neinštalovalo.

Po každom reštarte host'a treba profil odobrať nanovo. Dôvod je KASLR (časť 5.2). Existujúci profil sa bez -f neprepíše — je to poistka proti tomu, aby sa omylom prepísal profil, ku ktorému patria už uložené snímky. Keď sa na to zabudne, nástroj to povie sám: NESULAD PROFILU a návratový kód 5.

7.4 Merania cez scripts/root_run.sh

Merania sa nespúšťajú ručne poskladanými príkazmi, ale cez jediný skript, ktorý sa v tomto projekte spúšťa pod rootom. Dôvod je uvedený v jeho hlavičke: každý root beh je v práci udalosť, ktorú treba vedieť zopakovať a doložiť, takže skript do každého výstupu zapíše commit repozitára, dátum, presný príkaz a SHA-256 binárky.

podpríkaz	čo urobí	kam to uloží
probe	pripojí sa k doméne a vypíše memsloty, RAM, vCPU, test čítania	data/results/probe_<stamp>.json
once	jedna snímka	data/raw/<stamp>_once/ + data/results/once_<stamp>.json
run	periodický zber	data/raw/<stamp>_run/ + data/results/run_<stamp>.json
validate	pozemná pravda PRED → jedna plná snímka → pozemná pravda PO	data/sessions/<stamp>_validate/

Typická postupnosť pre ukážku:

```
sudo scripts/root_run.sh probe          # vidno doménu a jej memsloty?
sudo scripts/root_run.sh run -w delta -i 5 -N 6 # šesť cyklov, perióda 5 s
python3 -m guestparse ps --snapshot data/raw/<stamp>_run \
    --profile profiles/debian12-6.1.0-42-cloud-amd64
python3 -m features perbin --snapshot data/raw/<stamp>_run
```

Root treba iba na tie dva riadky so sudo, teda na samotné čítanie pamäte. Všetko ostatné — libvirt session, guest agent, ssh — beží pod bežným používateľom a skript sa doňho sám prepne (\$SUDO_USER). --dry-run vypíše, čo by sa spustilo, a skončí; root na to netreba.

Drobnosť, ktorá mátie: „probe nič nezapisuje“ platí o binárke. Skript okolo nej data/results/probe_<stamp>.json zapíše.

7.5 Čo pripraviť na živú ukážku (a dve pasce)

Pasca č. 1: root_run.sh validate porovnanie nespustí. Skript pozemnú pravdu odoberie a snímku urobí, ale validate.json v relácii obsahuje iba časy a výpisy. Samotné porovnanie treba spustiť ručne:

```
python3 -m guestparse validate \
  --snapshot data/sessions/<stamp>_validate/snap \
  --profile profiles/debian12-6.1.0-42-cloud-amd64 \
  --ps-before data/sessions/<stamp>_validate/ps_before.txt \
  --ps-after data/sessions/<stamp>_validate/ps_after.txt \
  --lsmod data/sessions/<stamp>_validate/lsmod_before.txt \
  --ss data/sessions/<stamp>_validate/ss_listen_before.txt \
  --out data/sessions/<stamp>_validate/porovnanie.json
```

Príručka to nehovorí a nezávislý posudok (fable-review.md) to našiel. Pri živej ukážke sa to prejaví okamžite: ukážeš validate a nič sa neporovná. Pozor aj na to, ktorý súbor pozemnej pravdy sa dá na --ss — ss_listen_* alebo ss_* mení význam stĺpcov (časť 5.8).

Pasca č. 2: features crosscheck bez prepínačov padal. docs/PRIRUCKA.md uvádza príkaz v tvare „features crosscheck ich porovná“, a posudok z 2026-09-19 zaznamenal, že takto padal s AttributeError: 'Namespace' object has no attribute 'sidecar' (features/cli.py:69). Prešlo to iba s prepínačmi --memslots a --c. Pri kontrole v ten istý deň neskôr ten istý príkaz prešiel s návratovým kódom 0, takže chyba bola medzitým opravená — a do práce sa uvádza aj s dátumom opravy, nie potichu.

Čo si pripraviť vopred. Profil musí byť z toho bootu, ktorý práve beží (inak kód 5). Adresár /var/tmp/vmic-val s validačnou snímku odomkne 21 preskočených testov. A jedna vec, ktorú komisia uvidí v logu tak či tak: pri načítaní BPF programu sa objaví nefatálne hlásenie libbpf: Error in bpf_create_map_xattr(pages): -EINVAL. Retrying without BTF. Je v stderr každého uloženého behu a je priznané v L12 (časť 9.2).

8. Čo máme namerané

Táto kapitola je zoznam čísel a zoznam dier. Obe sú rovnako dôležité. Komisia sa nepýta len „čo vám vyšlo“, ale aj „ako viete, že to vyšlo“ a „čo ste nezmerali“. Odpoveď na všetky tri musí byť pripravená vopred.

8.1 Ako sa tieto tabuľky čítajú

Každé číslo nižšie má tri veci: **hodnotu**, **n** (koľkokrát sa meralo) a **artefakt** — súbor v data/results/, z ktorého sa dá znova prečítať. Číslo bez artefaktu sa v práci neuvádza; to je pravidlo P5 v HONESTY.md. Veličina, ktorá sa nezmerala, má stav UNVERIFIED a je vypísaná v docs/LIMITACIE.md, obmedzenie L12.

Tam, kde v tomto dokumente stojí „**vlastný beh 2026-09-19**“, ide o príkaz spustený pri písaní tohto textu, ktorý ešte uložený artefakt v data/results/ nemá. Také číslo sa dá zopakovať uvedeným príkazom, ale do textu práce patrí až po tom, ako ho root_run.sh alebo stamp_results.py zapíše do data/results/.

Dve upozornenia na artefakty, ktoré ešte nie sú v gite. git status dnes hlási ?? data/results/probe_20260919T142620Z.js a ?? data/results/probe_20260919T142627Z.json — tieto dva súbory sú na disku, ale necommitnuté. Pred odovzdaním ich treba commitnúť, inak sú čísla, ktoré sa o ne opierajú, bez dohľadateľného artefaktu. Podobne data/sessions/20260919T120148Z_validate/porovnanie.json leží mimo data/results/; pravidlo „artefakt = súbor v data/results/“ na neho doslova nesedí, takže sa cituje s plnou cestou a s poznámkou, že je to výstup relácie, nie odvodený výsledok.

Prečo **medián** a nie priemer: priemer zdvihne jeden pomalý beh (napríklad keď hostiteľ práve niečo kompiloval). Medián je prostredná hodnota zoradeného radu a jeden výkyv ho nepohne. **p95** je hodnota, pod ktorou leží 95 % meraní — hovorí, aký zlý je zlý prípad, nie priemerný. **IQR** (interquartile range, medzikvartilové rozpätie) je rozdiel medzi 75. a 25. percentilom, teda šírka pásma, v ktorom leží prostredná polovica meraní; používa sa ako miera rozptylu, ktorá nie je citlivá na ojedinelé výkyvy. V tejto práci sa IQR zatiaľ nikde neuvádza — spomína sa iba ako súčasť metodiky, ktorá by uzavrela nezmerané veličiny z L12.

Značka éry agenta. Časť meraní z 18. septembra vznikla, keď v hosťovi ešte bežal agent zo staršieho projektu hypTcn002 (proces hypTcn_guest_ag, pid 376). Je to doložené v uloženej pozemnej pravde (data/results/2026-09-18_prvy_beh/ground_truth/ps_before.txt, riadok 68) a rozobraté v docs/MERANIA.md.

Agent bol zastavený 2026-09-18 o 15:29:20Z (Stopped hyptcn-guest-agent.service). **Referenčné sú merania po tomto čase.** Aby sa to dalo čítať z riadkov, používa tento dokument dve značky:

- **(A)** — meranie vzniklo, keď agent v host’ovi ešte bežal. Konkrétne sem patrí riadok vmic-val z 14:39–14:40 v časti 8.5 a čísla z adresára 2026-09-18_prvy_beh.
- **(Z)** — meranie vzniklo na zmrazenom host’ovi bez agenta, teda po 15:29:20Z. Sem patria merania z 2026-09-18_zmrazeny_host, optimalizácia hashu z 19:0x aj všetko z 19. septembra.

Prečo to vadí: pid 376 bol súčasťou stabilnej množiny procesov, takže „80/80“ z riadku (A) zahŕňa aj proces staršieho projektu.

8.2 Prostredie merania

Bez tejto tabuľky sú časy z častí 8.6 a 8.7 neinterpretovateľné. Údaje sú z hlavičiek artefaktov, nie z dnešného stavu stroja.

vrstva	položka	hodnota	zdroj
hostiteľ	OS	Fedora release 43 (Forty Three)	optim_hash_20260918.json:host.os
hostiteľ	jadro	7.1.13-100.fc43.x86_64	optim_hash_20260918.json:host.kern
hostiteľ	CPU	12th Gen Intel Core i7-12650H	optim_hash_20260918.json:host.cpu
hostiteľ	logických CPU	16	optim_hash_20260918.json:host.npr
hostiteľ	inštrukcie SHA-NI	áno	optim_hash_20260918.json:host.cpu_
hostiteľ	RAM	16 355 631 104 B	2026-09-18_zmrazeny_host/summary.json:host
hostiteľ	výstupný súborový systém	/var/tmp, ext4 na /dev/nvme0n1p3	optim_hash_20260918.json:host.vyst
hostiteľ	QEMU / libvirt	QEMU 10.1.5 (qemu-10.1.5-1.fc43), libvirt 11.6.0	docs/PRIRUCKA.md, kap. 7
hostiteľ	Python / torch	Python 3.14.7, torch 2.11.0+cu130 (CPU)	latency_score_20260919.json:host
host’	distribúcia	Debian GNU/Linux 12 (bookworm), 12.13	2026-09-18_zmrazeny_host/summary.json:gues
host’	jadro	6.1.0-42-cloud-amd64, Debian 6.1.159-1 (2025-12-30)	linux_banner zo snímky
host’	vCPU	2	probe_20260919T142627Z.json:values
host’	RAM	2 164 396 032 B (2,02 GiB), 528 417 stránok	probe_*.json:values.ram_bytes
host’	memsloty / efektívne oblasti	10 / 5	probe_*.json:values.memslots, stderr behov
host’	najvyššia fyzická adresa	0x100000000 (4 GiB)	probe_*.json:values.max_paddr
host’	libvirt	qemu:///session, doména hyptcn-guest	hlavičky artefaktov

Dôležité pre interpretáciu: výstup zberu ide na NVMe s ext4, takže časy zápisu v časti 8.6 sú časy NVMe, nie rotačného disku. A hostiteľ má 16 logických CPU, takže bežiaci zberač nesúťaží s VM o jediné jadro — čo je zároveň dôvod, prečo z týchto meraní nemožno robiť závery o vplyve na host’a (časť 8.8).

8.3 Zber: pripojenie k hypervízoru a rýchlosť čítania

Podpríkaz vmicollect probe je najkratší dôkaz, že sa modul naozaj napojil na KVM a vidí mapu memslotov. V data/results/ sú **štyri** artefakty probe:

veličina	hodnota	n	artefakt
memslotov domény hyptcn-guest	10	4	probe_20260918T154840Z.json, probe_20260919T004027Z.json, probe_20260919T142620Z.json, probe_20260919T142627Z.json
pamäť domény	2,02 GiB (2 164 396 032 B)	4	tie isté
vCPU	2	4	tie isté
najvyššia fyzická adresa	0x100000000 (4 GiB)	4	tie isté

Test čítania (vždy 16 646 144 B) dal štyri rôzne hodnoty:

dátum a čas	trvanie	rýchlosť	značka	artefakt
2026-09-18 15:48Z	6,6 ms	2394 MiB/s	(Z)	probe_20260918T154840Z.json
2026-09-19 00:40Z	12,3 ms	1290 MiB/s	(Z)	probe_20260919T004027Z.json
2026-09-19 14:26Z	14,4 ms	1103 MiB/s	(Z)	probe_20260919T142620Z.json
2026-09-19 14:26Z	7,3 ms	2164 MiB/s	(Z)	probe_20260919T142627Z.json

Rozptyl **1103–2394 MiB/s** je veľký a jeho príčina zmeraná nie je. Nie sú to opakovania toho istého merania: behy delí čas aj záťaž hostiteľa. Na obhajobe sa uvádza rozsah, nie najlepšie číslo. Dve veci k tomu patria:

- **Retrahované číslo:** 2394 MiB/s nahradilo skoršie „2401 MiB/s“, ktoré nepochádza zo žiadneho uloženého behu (retrakcia je zapísaná v docs/MERANIA.md, záznam z 2026-09-18, bod 1).
- **Test čítania v probe nie je priepustnosť plnej snímky.** Je to jedno krátke čítanie 16,6 MB. Poznámku, že test čítania nie je priepustnosť plnej snímky, nesie data/results/2026-09-18_zmrazeny_host/summary.json (kľúč poznamka_test_citania); samotné probe_*.json ju neobsahujú.

Periodický zber, zmrazený host' (Z), writer delta, perióda 5 s:

veličina	hodnota	n	artefakt
dokončených cyklov	12	1 beh	2026-09-18_zmrazeny_host/run_delta_5s.json
zlyhaných / zmeškaných slotov	0 / 0	—	ten istý
najväčšie meškanie	0,0 s	—	ten istý
plná snímka (cyklus #0)	čítanie 922,0 ms, zápis 1638,7 ms, spolu 2560,8 ms; 123 845 z 528 417 stránok	1	2026-09-18_zmrazeny_host/summary.json
trvanie delta cyklu	medián 564,5 ms, rozsah 552,6–629,6 ms	11	ten istý
zmena pamäte medzi snímkami, všetky delta cykly	0,040–0,659 % z 528 417 stránok	11	ten istý
zmena pamäte, iba cykly bez prihlásenej SSH relácie	0,040–0,050 % (212–265 stránok)	7	ten istý, kľúč prirastkove_snimky_bez_sedenia_ssh
pauza VM	0,0 ms, paused=false	79 sidecarov	vlastný sken data/**/*.json, schéma vmicollect/1

Horná hranica 0,659 % nie je šum: v cykloch #4 až #7 (16:32:37–16:32:52Z) sa do host'a prihlásil a odhlásil ssh a ukončovala sa user@0.service. Samotné meranie je viditeľná činnosť. Artefakt tieto cykly vedie oddelene práve preto.

Novšie behy binárkou z repa potvrdzujú spoľahlivosť plánovača: run_20260919T110017Z.json (10 cyklov) a run_20260919T112521Z.json (8 cyklov), obe cycles_failed 0, cycles_skipped 0, worst_lateness_s 0.0 (Z).

8.4 Príznakový vektor: C proti nezávislej Python referencii

Ten istý per-bin vektor počíta modul v C (vmicollect/src/perbin.c) a nezávisle od neho referencia v Pythone (features/perbin.py). Keby sa líšili, v datasete by boli čísla z dvoch implementácií a späťne by sa nedalo zistiť, ktorý riadok je z ktorej.

veličina	hodnota	artefakt
porovnaných snímok $\times$ binov $\times$ polí	$6 \times 131 \times 8 = \mathbf{6288 \text{ polí}}$	perbin_crosscheck_20260918.json
rozdielov nad toleranciou $5 \cdot 10^{-7}$	0	ten istý, porovnanie.rozdielov
najväčší skutočný rozdiel	$4,84 \cdot 10^{-7}$	ten istý,
changed_ratio		porovnanie.max_abs_rozdiel
polí mimo tlačenej presnosti (6 desatinných miest)	0	ten istý,
		porovnanie.mimo_tlacenej_presnosti

Invarianty sedia aj mimo binov: súčet pages_changed a pages_total zo sidecaru sa rovná súčtu z referencie na všetkých šiestich snímkach.

8.5 Rekonštrukcia objektov host'a proti pozemnej pravde

Pozemná pravda znamená: v host'ovi sa pred snímkou a po nej spustí ps, lsmod a ss, výpisy sa uložia, a to, čo parser vytiahol zo snímkov, sa porovná s nimi. Porovnáva sa **stabilná množina** — procesy, ktoré sú v oboch výpisoch ps. Proces, ktorý medzitým vznikol alebo skončil, sa nepočíta ani do chýb. Ktorý výpis ss je pozemnou pravdou, mení význam stĺpca — pozri časť 5.8.

dátum / relácia	značka	procesy	moduly	sokety (pozemná pravda)	sedí / chýba / navyše	artefakt
2026-09-18 14:39–14:40, vmic-val	(A)	80 / 80 = 100 %	neporovnané (compared:false; zo snímky 47)	neporovnané (compared:false; zo snímky 13)	—	validate_20260918_vmic-val.json
2026-09-18 15:49, relácia 154914Z, ss_before	(Z)	80 / 80	51 / 51	ss -tuanp, 12	11 / 1 / 1	validate_20260918T154914Z
2026-09-18 15:49, relácia 154914Z, ss_after	(Z)	80 / 80	51 / 51	ss -tuanp, 12	10 / 2 / 2	validate_20260918T154914Z
2026-09-18 16:33, zmrazený host'	(Z)	81 / 81	51 / 51	ss -tulpn, 10	10 / 0 / 2	2026-09-18_zmrazeny_host/validate
2026-09-19 12:01, relácia 120148Z	(Z)	78 / 78 = 100 %	50 / 50	ss -tulpn, 10	10 / 0 / 2	data/sessions/20260919T120148Z

Vo všetkých riadkoch: chýbajúcich procesov 0, falošných 0, nezhoda mien 0, prechod zoznamov neprerušovaný. Zhoda mien nie je obyčajné porovnanie reťazcov — ps a pole task_struct.comm sa líšia pri jadrových workeroch a pri

menách dlhších než 15 znakov. Pravidlá sú v `guestparse/validate.py` a ich počty sa ukladajú: v behu z 19. 9. exact 63, worker 12, truncated 3; v behu na zmrazenom host'ovi exact 63, worker 15, truncated 3.

Riadok `vmic-val` má navyše druhú zvláštnosť: `ps_before` 84 a `ps_after` 82, teda medzi výpismi sa počet procesov v host'ovi zmenil, a práve preto je stabilná množina 80.

Vlastné behy 2026-09-19 nad snímkou `data/raw/20260919T110155Z_once`: krížová kontrola prekladu adries zhoda pre `linux_banner` aj `init_task`; `ps` 78 procesov; `lsmod` 47 modulov; `ss` 13 soketov; `checks` 451 položiek `sys_call_table`, 0 nálezov, všetky tri kontroly uzavreté, návratový kód 0. K tejto konkrétnej snímke pozemná pravda uložená nie je, takže sú to výpisy, nie validácia.

8.6 Optimalizácia: kontrolný súčet pred a po

Pôvodný raw writer počítal SHA-256 až na konci, nad hotovým **riedkym** súborom.

Čo je riedky (sparse) súbor. Súborový systém nemusí na disku držať bloky, do ktorých sa nikdy nezapísalo — namiesto nich si zapamätá „tu je diera“ a pri čítaní vráti nuly. Súbor sa teda tvári, že má 4 GiB, pričom na disku zaberá len to, čo sa naozaj zapísalo. Problém bol, že SHA-256 nad takým súborom musí **prečítať aj tie diery** (čiže vyrobiť gigabajty núl a prehnať ich cez hash), hoci sa nikdy nezapísali.

Oprava: hash sa počíta priebežne nad tým, čo sa naozaj zapisuje (vo funkcii `feed()`), plus vetva **SHA-NI** — inštrukcie procesora určené priamo pre SHA-256.

variant	čítanie (medián)	zápis (medián)	cyklus spolu (medián)	n
pred, hash=sha256	388,1 ms	12 983,7 ms	13 364,9 ms	3
pred, hash=none	387,4 ms	206,8 ms	589,9 ms	3
po, hash=sha256	2287,9 ms	196,2 ms	2480,9 ms	3
po, hash=none	380,7 ms	198,0 ms	577,7 ms	3

Zrýchlenie celého cyklu je $13\,364,9 / 2\,480,9 = 5,4\times$ (`optim_hash_20260918.json`, surové behy v `optim_hash_20260918_surove.značka (Z)`). Mikrobenchmark samotného hashu nad 1 GiB: **2150,7 MiB/s** s SHA-NI proti **357,3 MiB/s** bez nich, teda $6,0\times$ (kľúč `sha256_priepustnost_mib_s`, `n = 3`).

Čo sa tým zhoršilo, a treba to povedať prvé: hashovanie sa presunulo do fázy čítania, takže okno, počas ktorého sa číta pamäť bežiackej VM, sa predĺžilo z 388,1 ms na 2287,9 ms. Snímka je živá, takže dlhšie okno znamená väčšiu šancu, že dve stránky pochádzajú z odlišnejších okamihov. Kto potrebuje krátke okno, má `output.hash=none`. Vplyv tejto výmeny na presnosť rekonštrukcie zameraný nie je.

Vedľajší dôsledok, ktorý artefakt sám zdôrazňuje: po oprave už `read_mib_s` nie je rýchlosť čítania pamäte, lebo hash je vnútri meraného úseku. Rýchlosť čítania sa meria s `output.hash=none`.

8.7 Latencie

Latencia snímka → príznakový vektor sa meria od začiatku cyklu zberu po zavretie sidecaru s blokom `features` na disku (cez `inotify_close_write`). Artefakt `latency_vector_20260918.json`, spolu `n = 78` cyklov, 0 zmeškaných slotov a 0 chýb vo všetkých šiestich behoch, značka (Z).

režim	n	medián	p95	min-max
príznačky zapnuté, 5 s, delta	24	605,2 ms	617,9 ms	594,7–630,3 ms
príznačky vypnuté, 5 s, delta	24	554,5 ms	558,7 ms	542,6–563,7 ms
príznačky zapnuté, 2 s, delta	12	604,3 ms	612,6 ms	596,4–618,7 ms
príznačky vypnuté, 2 s, delta	12	555,1 ms	561,1 ms	543,6–562,7 ms
príznačky zapnuté, 2 s, plná snímka	1	1588,0 ms	—	—

režim	n	medián	p95	min–max
príznačky zapnuté, 5 s, plná snímka	2	1595,7 ms	—	1584,7–1606,7 ms

Falzifikovateľné kritérium použiteľnosti: p95 latencie cyklu < perióda zberu. Pri perióde 2 s je p95 delta cyklu 612,6 ms, teda rezerva asi 3×. Plná snímka pri perióde 2 s trvá 1588 ms — stále pod periódou, ale rezerva je malá.

Cena výpočtu príznakov (modul si ju meria sám ako `compute_ms`): delta cyklus medián 52,4 ms ($n = 24$, perióda 5 s) a 51,5 ms ($n = 12$, perióda 2 s); plná snímka 478,5–482,9 ms s entropiou ($n = 2$). Entropia stojí 3,498 μ s na zmenenú stránku (`data/results/perbin_c_20260918/perbin_c_20260918.json`, kľúč `entropia_us_na_zmenenu_stranku`, $n = 6$), preto je samostatný prepínač.

Latencia snímka → skóre, meraná nad uloženou snímkou (`latency_score_20260919.json`). Pozor na to, že tento artefakt obsahuje **tri rôzne merania času modelu** a nesmú sa zamieňať:

fáza	n	medián	p95	kľúč v artefakte
výpočet príznakového vektora	10	661,8 ms	709,3 ms	<code>values.vektor_ms</code>
prvý dopredný beh modelu v procese	10	1,728 ms	2,352 ms	<code>values.model_ms_prvy_v_proces</code>
ustálený dopredný beh, okno dĺžky 1	100	0,486 ms	0,789 ms	<code>values.model_ms.okno_1</code>
ustálený dopredný beh, okno dĺžky 16	100	0,301 ms	0,640 ms	<code>values.model_ms.okno_16</code>
spolu (vektor + prvý beh modelu)	10	663,6 ms	711,1 ms	<code>values.spracovanie_ms</code>

Rozdiel medzi 1,728 ms a 0,486 ms nie je chyba merania: prvý dopredný beh v procese je drahší kvôli inicializácii torchu, čo artefakt sám hovorí v kľúči `model_ms.poznamka`. Skórovací proces beží dlhodobo, takže sa tento čas platí raz. **Nemieša sa n = 100 s hodnotou 1,728 ms** — to je hodnota s $n = 10$.

Podmienky, za ktorých to vzniklo, sú v artefakte a patria k číslu: meralo sa nad jednou uloženou **plnou** snímkou (`data/sessions/20260919T004123Z_validate/snap`) s oknom dĺžky 1, zber pritom nebežal, a `loadavg` hostiteľa bol 2,7–3,4, takže namerané časy sú horný odhad. Ukážkový výstup softmaxu bol vo všetkých behoch [0.66303, 0.33697] (seed aj vstup rovnaké) — a nehovorí nič o stave VM.

Vlastný beh 2026-09-19 (`python3 -m tcn.score --snapshots data/raw/20260919T110017Z_run --profile profiles/debian12-6.1.0-42-cloud-amd64 --dlzka 4`): spracovaných 10 snímok, z toho 7 so skóre (prvé tri okno ešte nenaplnia). Výpočet vektora: prvá snímka 434–452 ms (studený cache; dva behy toho istého príkazu), ďalších deväť 180–194 ms. Model: prvá snímka 1,8–2,4 ms, ďalšie 0,7–1,2 ms. Prvé hodnoty sa medzi behmi líšia, ustálené sedia. Skóre [0.8521, 0.1479] na prvej skórovanej snímke. Rozdiel voči 661,8 ms z artefaktu je stav vyrovnávacej pamäte stránok a menšie snímky, nie zlepšenie kódu. Tento beh zatiaľ nemá uložený artefakt v `data/results/`.

Živá latencia snímka → skóre, teda počas bežiaceho zberu, nemá uložený artefakt. Je to UNVERIFIED v L12 a presný príkaz, ktorý ju zmeria, je zapísaný priamo v artefakte pod kľúčom `co_sa_NEMERALO`. Formulácia je tu dôležitá: nezávislý posudok (`fable-review.md`) uvádza, že ju **zmeral** — `tcn.score` — cakaj nad adresárom bežiaceho run dal medián 1052 ms a p95 1264 ms pri $n = 5$ a perióde 5 s. Artefakt v `data/results/` k tomu nie je, takže stav UNVERIFIED obostojí, ale veta musí znieť „nie je uložený artefakt“, nie „zmeraná nie je“ — inak ju posudok vyvracia.

8.8 Čo nemáme

Natréňovaný model a presnosť detekcie. Model má náhodné váhy so seedom 0. Dôvod je jediný a nie je technický: na stroji, kde práca vznikla, nie sú reálne malvérové vzorky. Podrobne v časti 6.8 a v L16.

Vplyv zberu na bežiacu VM. Nezmerané, UNVERIFIED. Chýba benchmark s pevne daným objemom práce v hosťovi, schéma A/B/A, medián a IQR z aspoň 12 opakovaní. Zmerané je len to, že VM sa nezastavuje (`pause_ms`

0,0, paused=false vo všetkých 79 uložených sidecaroch). To nie je to isté ako „bez vplyvu“: čítanie ide cez `bpf_copy_from_user_task()`, a pri non-anonymne podloženej pamäti to nedotknutú stránku hostiteľovi **naozaj alokuje**. Zberač na to sám varuje; v meraných behoch išlo o 4,00 KiB podložených cez `/dev/zero` (deleted) (L2).

Vplyv na hostiteľa, dlhý beh, druhá VM, druhý profil. Ani jedno. Spotreba CPU a pamäte procesu zberača sa nemerala. Beh rádovo v hodinách s retenciou nebežal. Všetky merania sú z jednej domény `hyptcn-guest` a z jedného profilu jadra. Periódy iné než 2 s a 5 s sa neskúšali.

Vplyv presunu hashu na presnosť rekonštrukcie. Predĺženie čítacieho okna z 388 ms na 2288 ms (časť 8.6) teoreticky zvyšuje riziko roztrhnutia zoznamov v živej snímke. Zmerané to nie je.

Text práce. Repozitár obsahuje kód, merania a dokumentáciu. Kapitoly diplomovej práce v ňom nie sú.

9. Obmedzenia L1 až L17

`docs/LIMITACIE.md` je zoznam vedomých obmedzení. Každé má v ňom štyri časti: **fakt**, **overenie** (príkaz a jeho skutočný výstup), **formuláciu do textu práce** a **čo by to odstránilo**. Zásada, ktorá tam stojí na začiatku, je zároveň odpoveď na otázku „prečo toľko priznávať“: priznaná limitácia je silnejšia pozícia než zamlčaná, lebo recenzent ju nájde tak či tak a rozdiel je iba v tom, či ju nájde napísanú, alebo objavenú.

Limitácia sa zo zoznamu neodstraňuje, kým nezmizne z kódu; keď zmizne, pripíše sa k nej dátum, commit a odkaz na meranie (to je prípad L14).

9.1 Zoznam

L1 — Živá snímka: VM sa nezastavuje. Backend `ebpf` nemá operáciu pause (`.pause = NULL`), takže sa stroj počas čítania nezastavuje a stránky zo začiatku a z konca snímky sú z rôznych okamihov. Dôsledok: spájaný zoznam jadra sa môže počas prechodu roztrhnúť, preto každý prechod hlási `truncated`.

L2 — Čítanie nie je úplne pasívne. Pri anonymne podloženej pamäti sa nedotknutá stránka prečíta ako nulová a hostiteľovi nič nepridruží; pri non-anonymnom podložení (`shmem`, `memfd`, `hugetlbfs`, súbor) ju čítanie na hostiteľovi **skutočne alokuje**. Zberač to deteguje z `/proc/<pid>/maps` a pri štarte varuje; v meraných behoch išlo o 4,00 KiB.

L3 — Profil jadra host'a je vstupná závislosť získaná z host'a. `callsyms` a `BTF` sa získali raz, z host'a, mimo behu zberu — rovnako ako profil pri `LibVMI` alebo `Volatility`. Veta „bez akejkoľvek znalosti host'a“ sa v práci nikde nepoužíva.

L4 — Parsuje sa pamäť jadra, nie pamäť procesov. Preklad pokrýva priestor jadra; obsah pamäte používateľských procesov (argumenty, halda, knižnice) sa neparsuje, lebo chýba `CR3` príslušného `vCPU` alebo `mm_struct->pgd`, a backend registre `vCPU` nečíta. Odstránilo by to rozšírenie BPF programu alebo prechod cez `task_struct->mm->pgd`; hotové ani zmerané to nie je.

L5 — hole_test netestuje BPF vetvu. Regresný test beží nad **falošným backendom** a overuje kontrakt `read_pa()` a spoločnú čítaciu slučku; vetva `KIND_HOLE` v samotnom BPF programe **v testoch nikdy nebežala**. Jediná opora pre ňu je, že v reálnych behoch nad `hyptcn-guest` bol počet chýb čítania nulový. (Dokument to podrobne rozoberá v časti 2.6; tvrdiť, že `hole_test` stráži BPF vetvu, je presne to preháňanie, ktoré L5 zakazuje.)

L6 — Injekčný test je injekcia do kópie snímky. Pozitívny prípad detekcie hookov sa robí tak, že sa urobí **kópia** reálnej snímky a v kópii sa prepíše jedna položka `sys_call_table` na adresu v oblasti modulov; v host'ovi sa nemení nič a žiadny rootkit sa nikam neinštaluje. Dôvod, prečo tam pozitívny prípad musí byť: kontrola, ktorá na čistom host'ovi vždy vráti nulu, je na nerozoznanie od funkcie, ktorá vždy vracia nulu. Dôvod, prečo to nie je skutočný rootkit: host' nemá `gcc`, `make`, hlavičky jadra ani funkčné DNS.

L7 — Confidential VM (SEV-SNP, TDX) sa takto prečítať nedá. Pamäť dôverného stroja je v `guest_memfd` a cez adresný priestor `VMM` tam jednoducho nie je. Kód pritom rozlišuje jemnosť: samotný príznak `KVM_MEM_GUEST_MEMFD` ešte neznamená nečitateľnosť — nečitateľný je až slot s `GMEM_ONLY` alebo bez `userspace_addr`.

L8 — Prenos cez `mmap` nie je „nulové kopírovanie“. Je to jedno prekročenie hranice jadro/používateľ plus jeden `memcpy` v používateľskom priestore na blok (`backend_ebpf.c:787 a:1128`). Tak sa to musí aj nazvať.

L9 — Hook je notifikačný. `vmic_hook_api_t` má tri polia (`abi`, konfigurácia iba na čítanie, `log`) a **neobsahuje** `read_pa` ani inú cestu k pamäti host'a. Plugin dostane cestu k hotovému súboru a časy; ak chce obsah, musí si súbor otvoriť sám. Beží synchrónne v hlavnej slučke, takže výpočet dlhší než perióda spôsobí zmeškávanie slotov. Podrobne v časti 4.2.

L10 — Kontroly integrity majú pomenované slepé miesta. Tri: hook prepísaný na adresu **vnútri** rozsahu `[_stext, _etext)` rozsahová kontrola z princípu nevidí; skrytie, ktoré odpojí uzol z oboch porovnávaných štruktúr naraz, nevytvorí rozdiel; a nulový nález z neuzavretej kontroly nič nedokazuje — preto návratový kód 4.

L11 — Jeden výstupný adresár na jeden zberač. Retencia pracuje nad celým `output.dir` a nerozlišuje domény, takže dva zberače v jednom adresári si navzájom mažu snímky. Každý doméne treba vlastný `output.dir`. Namerané výsledky to neovplyvňuje, nasadenie áno. Podrobne v časti 4.5.

L12 — Čo ešte nie je zmerané. Päť veličín so stavom `UNVERIFIED`: živá latencia snímka → skóre, vplyv zberu na výkon vnútri host'a, vplyv zberu na hostiteľa, dlhý beh s retenciou a správanie pri periódach iných než 2 s a 5 s. K tomu presnosť voči reálnym malvérovým vzorkám so stavom „**nebude zmeraná**“, lebo vzorky nie sú dostupné a nenahrádzajú sa ničím. Tu je aj priznané nevyriešené hlásenie `libbpf` (časť 9.2).

L13 — Príznakový vektor: čo o ňom treba vedieť pred tréningom. Na plnej snímke sú `changed_ratio` a `zero_ratio` **kolineárne** — platí presne `changed_ratio = 1 - zero_ratio`, lebo tabuľka hashov je na začiatku reťazca nulová a „zmenená“ tam znamená „nenulová“ (overené na `sidecari seq 0: 0,854823 + 0,145177 = 1,000000` na všetkých binoch). Riadok plnej snímky má teda iný význam než riadky `delta` snímok; preto `je_plna`. K tomu: výpočet príznakov predlžuje cyklus o 51–53 ms, entropia o 3,498 µs na zmenenú stránku.

L14 — Procesy, moduly a sokety do príznakového vektora nevstupujú — VYRIEŠENÉ 2026-09-19. Obmedzenie už neplatí; zo zoznamu sa neodstraňuje, aby nevyzeralo, že nikdy nebolo. Vyriešil ho `features/snapshot.py` (commit `da8ad2e`), ktorý zloží 20 príznakov plus `ma_predchodcu` a `je_plna` do 22 stĺpcov, pričom objektová časť sa číta cez `guestparse` nad **tou istou** snímkou.

L15 — Časové rozlíšenie: proces kratší než perióda zberu je neviditeľný. Toto je priama odpoveď na otázku o „real-time“ a patrí k najdôležitejším. Metóda je snímková: vidí len to, čo v host'ovi existuje v okamihu snímky. Doklad je meraný: v sedení `data/raw/20260919T011815Z_fork_storm` bežalo v host'ovi 10 sekúnd vetvenie procesov, ktoré podľa vlastného súhrnu vytvorilo a pozbieralo **8320 detí v 416 dávkach**; zber mal periódu 5 s a urobil 2 snímky. Príznačky `proc_new`, `proc_gone` a `mod_delta` zostali v oboch riadkoch na nule: množina PID sa medzi oboma snímkami nezmenila ani o jeden prvok, hoci v tom čase v host'ovi vzniklo a zaniklo 8320 procesov. Zatiaľ čo `mem_changed_ratio` medzi snímkami kleslo z 0,2647 na 0,0137, takže pamäťová časť vektora rozdiel vidí a objektová nie. Nie je to chyba implementácie a lepším parsovaním sa to neodstráni: horná hranica je daná periódou. Odstránil by to iba udalostný zdroj (hook na `execve` alebo na plánovač), teda iná vrstva — a hook je tu notifikačný (L9).

L16 — Presnosť detekcie malvéru nebola meraná a v tejto verzii sa merať nedá. Bod zadania „vyhodnotenie presnosti detekcie malvérových vzorcov“ je **nesplnený** a dôvod je vstup, nie implementácia. Vetva zberu korpusu bola odstránená (časť 9.3).

L17 — Profil jadra host'a platí pre jeden boot, nie pre verziu jadra. `KASLR` posunie adresy pri každom štarte; doména bola 18. 9. vypnutá a 19. 9. naštartovaná a `init_task` sa presunul z `0xffffffff97a1aa40` na `0xffffffffb221aa40`. So starým profilom sa posun jadra síce nájde (banner sa v pamäti naskenuje), ale prechod tabuľkami stránok zlyhá a nástroj to od 19. 9. hlási kódom 5. Rovnaká závislosť platí pre `LibVMI` aj `Volatility`.

9.2 Nevyriešené hlásenie `libbpf`

Pri každom načítaní BPF programu sa v `stderr` objaví:

```
libbpf: Error in bpf_create_map_xattr(pages): -EINVAL. Retrying without BTF.
```

Je to **prvý riadok** `stderr` v každom uloženom behu — overené dnes v `run_20260919T110017Z.json`, `run_20260919T112521Z.json` aj `2026-09-18_zmrazeny_host/run_delta_5s.json`. Mapa `pages` sa vytvorí aj bez BTF a zber funguje bez ďalších následkov, ale **príčina zistená nebola**. L12 to vedie medzi priznanými nedoriešenými vecami.

Prečo to patrí do dokumentu: komisia `log` uvidí a slovo `Error` v ňom je. Odpoveď má znieť „je to nefatálne, mapa vznikne bez BTF, zber tým nie je dotknutý, príčinu sme nezistili a je to zapísané v L12“ — nie prekvapenie.

9.3 Vetva zberu korpusu bola zmazaná

Toto treba vedieť skôr, než sa niekto spýta na šesť scenárov, o ktorých sa píše v častiach 6.8 a 8.8.

Adresár `scenarios/` a skript `scripts/collect_corpus.sh`, ktoré scenáre správania spúšťali v host'ovi a označovali sedenia, boli z pracovného stromu **odstránené** (L16, commit `da8ad2e`). Overené dnes: `ls scenarios` aj `ls scripts/collect_corpus.sh` vracajú „No such file or directory“. V histórii gitu kód zostáva.

Dôsledok: v `data/raw/` ležia dve označené sedenia — `20260919T011728Z_mass_file_rewrite` (v host'ovi 200 súborov, 50 281 zápisov, 3142,56 MiB, trvanie 24,766 s) a `20260919T011815Z_fork_storm` (8320 detí v 416 dávkach, trvanie 10,002 s). Obe majú `labels.json` so sha256 scenára, ktorý ich vyrobil, ale samotný scenár v strome nie je. **Dnes sa teda nedajú reprodukovať** a nič ich nespracúva; v práci slúžia len ako doklad k L15, nie ako dáta.

Formulácia pre obhajobu: scenáre napísané boli a dve sedenia existujú, ale na tréning sa vedome nepoužili a vetva bola zo stromu odstránená, aby sa z nich omylom nestal korpus.

10. HONESTY.md: pravidlá, strojová kontrola a prečo sa projekt začal nanovo

10.1 Prečo tento súbor vôbec existuje

HONESTY.md je záväzný pre všetok text a kód v repozitári. Jeho prvá veta hovorí dôvod: predchádzajúca iterácia projektu (`hypTcn002`) vyprodukovala čísla, ktoré neobstáli pri kontrole — a nešlo o preklep, ale o **systematickú chybu v metodike**. Pravidlá sú zápisom toho, čo sa nesmie zopakovať.

10.2 Dvanásť pravidiel

pravidlo	čo hovorí
P1 — zakázané čísla	Čísla zo zoznamu v časti 10.4 sa nikdy neuvádzajú ako výsledok tejto práce; smú sa objaviť iba na riadku so slovom RETRAKCIA, s commit hashom a vetou, čo ich zneplatnilo.
P2 — ilustračné čísla nie sú merania	Čísla v <code>vmicollect/README.md</code> a v hlavičkových komentároch zdrojákov sú príklady formátu alebo rádové odhady, nie namerané hodnoty; do textu práce sa neprepisujú.
P3 — každé číslo má značku zdroja	Každé číslo v <code>thesis/</code> musí mať značku <code>{{res:subor.json:kluc}}</code> ukazujúcu na súbor v <code>data/results/</code> . Čierna listina sama nestačí — tá zachytí iba čísla, ktoré už poznáme.
P4 — detekčné výsledky sa volajú syntetické scenáre správania	Triedy sa pomenúvajú podľa správania (<code>cpu_burn</code> , <code>mass_file_rewrite</code> , ...), nie podľa rodín malvéru, a veta „presnosť voči reálnym malvérovým vzorkám nebola meraná“ patrí do abstraktu, do kapitoly o testovaní aj do záveru.
P5 — „funguje“ vyžaduje výstup behu z tohto repa	Tvrdenie o funkčnosti platí len vtedy, keď existuje uložený výstup behu binárky z tohto repozitára. Čo nikdy nebežalo, je UNVERIFIED. Tri menovite zakázané prehánania: <code>hole_test</code> netestuje BPF vetvu <code>KIND_HOLE</code> , <code>hook API</code> je notificačné, prenos cez <code>mmap</code> nie je „nulové kopírovanie“.
P6 — profil jadra host'a je priznaná vstupná závislosť	Nikde sa nepíše „bez akejkoľvek znalosti host'a“. Rozlišuje sa <code>qemu-guest-agent</code> (orchestrácia experimentu a pozemná pravda, priznané) od bezpečnostného agenta v host'ovi (protirečí zadaniu, nepoužíva sa, pred meraniami sa ukončí a zaznamená sa to).

pravidlo	čo hovorí
P7 — nemeranú vlastnosť nemožno nazvať vlastnosťou	„Minimálny vplyv na VM“ sa smie napísať len s číslom a intervalom spoľahlivosti z merania. „Real-time“ sa nahrádza konkrétnou periódou zberu a nameranou latenciou. Slovo bez čísla sa škrtá.
P8 — negatívny výsledok sa uvádza	Keď baseline dorovná alebo prekoná TCN, je to v tabuľke aj v závere. Keď padne confound test, výsledok sa označí za neplatný a vysvetlí sa. Labely sa po vyhodnotení nemenia.
P9 — provenience prevzatého kódu	Každý prevzatý kus kódu má pôvod uvedený v hlavičke súboru a v docs/ARCHITEKTURA.md. Veta „autorstvo vmicollect sa uvádza ako neznáme, kým sa nezistí“, ktorá je v tomto pravidle dnes zapísaná, je už prekonaná — pôvod sa zistil a je v časti 3.9.
P10 — docs/MERANIA.md je append-only	Meranie sa nikdy neprepisuje; oprava alebo retrakcia je nový záznam s dátumom, ktorý odkazuje na pôvodný. Platí to aj pre data/results/*.json: súbor sa nemení, pridáva sa nový.
P11 — zakázané slová	<i>state-of-the-art, robustný, komplexný, inovatívny, unikátny, production-ready</i> , a na žiadosť zadávateľa navyše <i>výrazne a revolučný</i> . Nahrádza sa číslom, alebo sa veta škrtne.
P12 — dokumentácia sa overuje proti kódu, nie naopak	Keď sa text a kód nezhodujú, platí kód a opraví sa text. Doložené prípady: dokumentácia uvádzala jadro host'a 6.1.0-44, kým bežalo 6.1.0-42; systemd unit bol inactive, hoci proces bežal; ARCHITECTURE.md v hypTcn002 popisoval Poissonovské plánovanie, kým kód mal pevný ticker.

10.3 Strojová kontrola scripts/check_claims.sh

Skript kontroluje thesis/, docs/, koreňový README.md, vmicollect/README.md a komentáre v zdrojákoch. V zdrojáku kontroluje **text komentárov**, nie kód: konštanta 0.86 v algoritme nie je metrika z hypTcn002, kým tá istá hodnota vo vete v komentári tvrdením je.

Kódy nálezov: ZAKAZANE-CISLO, BEZ-ZNACKY, ZNACKA-TVAR, ZNACKA-SUBOR, ZNACKA-KLUC, ZNACKA-HODNOTA, ZAKAZANE-SLOVO, JSON-HLAVICKA. Značka má tvar `{{res:subor.json:kluc}}` a píše sa hneď za číslo, na ten istý riadok, aj s jednotkou. Skript overí, že súbor existuje, že kľúč v ňom existuje, a že číslo napísané pred značkou sa po zaokrúhlení zhoduje s hodnotou v JSON. **Keď sa nezhoduje, opraví sa text, nikdy JSON.**

Výnimky sú vždy stopa v texte, nie tichý zoznam vedľa: RETRAKCIA na riadku, `<!-- noclaim: dôvod -->`, bloky `<!-- prevzate:start -->...<!-- prevzate:end -->` a verzionovaný thesis/.claims-allow. Jediná výnimka mimo textu je pole EXEMPT v samotnom skripte a uplatnené výnimky sa na konci každého behu vypíšu.

Vlastný beh dnes:

```
$ scripts/check_claims.sh
check_claims.sh: uplatnene vynimky (pole EXEMPT, pozri HONESTY.md kap. 3):
  vmicollect/README.md: 1043.2 - vymysleny priklad JSON sidecaru ...
  vmicollect/README.md: 2,8 TB - ilustracny radovy odhad ...
  vmicollect/src/retention.c: 2.8 TB - ten isty radovy odhad ...
  vmicollect/src/writer_delta.c: 2.8 TB - ten isty radovy odhad ...
check_claims.sh: ciste (94 suborov)
```

Čo kontrola **nezachytí**, je napísané v samotnom HONESTY.md a treba to vedieť povedať: nepravdivú vetu bez čísla, číslo prepísané aj v JSON aj v texte, zlú metodiku za správne uloženým číslom, číslo v kóde mimo komentára, a prítomnosť hlavičky ako záruku, že sú v nej pravdivé údaje. Skript je sito na to, čo sa dá overiť textovo — nie dôkaz poctivosti.

HONESTY.md sa zámerne nekontroluje sám sebou: obsahuje zoznam zakázaných čísel, takže by hlásil sám seba.

10.4 Zakázané čísla z hypTcn002 a prečo sa projekt začal nanovo

Všetky pochádzajú z predchádzajúcej iterácie. **Spoločný dôvod neplatnosti je confound zo session-level labelov:** každé okno v rámci jednej session dostalo label podľa toho, čo sa v tej session spúšťalo, a benígne aj „malvérové“ sessions sa líšili úrovňou aktivity systému. Okná z jednej session sa navyše miešali medzi tréningovú a testovaciu množinu. Model sa naučil rozlíšiť vytiažený stroj od nečinného, nie malvér od benígneho softvéru.

Prečo sa to nedá „opraviť prepočítaním“: číslo je neplatné preto, že neplatí experiment, ktorý ho vyrobil. Doklad je priamo v zozname — po oprave splitu metriky klesli na 0,86, ale confound zostal, takže neplatné sú aj tie.

číslo	čo to malo byť	prečo je neplatné
F1 = 99,70	F1 binárnej detekcie	session-level labels + miešanie okien medzi split
AUC = 99,98	ROC AUC binárnej detekcie	to isté
0,9997; 0,9995; 0,9998; 0,9984	metriky na okno	to isté
0,9869; 0,9861; 0,9956	metriky na okno	to isté
95,5 % (5 tried)	presnosť viactriednej klasifikácie	to isté, navyše trieda = session
0,86; 0,8583; 0,8633	metriky po oprave splitu	opravený bol split, nie labels; confound zostal
0,999	AUC	to isté
LogReg 0,852	baseline	to isté; baseline dorovnal model, čo bol sám o sebe signál confoundu
26/33 (78,8 %)	detegované vzorky	pomer zo vzoriek, ktoré neboli overené ako spustené
SPRT p = 1,75e-12	sekvenčný test	p-hodnota z okien, ktoré nezávisle neboli
overhead 1,76 %	vplyv na VM	merané iným senzorom (in-guest eBPF agent), inou metodikou, bez CI
4,08 f/s	priepustnosť	iný senzor
3,4 ms	latencia	iný senzor, bez n a bez rozdelenia
0,741 ± 0,020	per-PID AUC po kontrole confoundu	jediné číslo, ktoré prežilo kontrolu — ale patrí inému senzoru a inému modelu, preto sa smie uviesť iba v kapitole o predchádzajúcich iteráciách, na riadku s RETRAKCIA

Zoznam je zámerne širší než to, čo by sa dalo obhájiť: keď sa niektoré z týchto čísel objaví v texte ako nový výsledok, je to zhoda, ktorú treba vysvetliť, nie prehliadnuť.

Odpoveď na otázku „prečo ste začali nanovo“ má znieť takto: predchádzajúca iterácia merala niečo iné, než tvrdila. Senzor bol iný (agent vnútri host'a, čo protirečí zadaniu o neinvazívnosti), labels boli na úrovni sedenia, a rozdiel medzi triedami bol rozdiel v zaťažení stroja. Opraviť sa dal split, nie experiment. Nová iterácia preto začína od senzora — čítanie zvonka cez KVM memsloty — a od pravidiel, ktoré takému číslu zabránia vzniknúť: artefakt ku každému číslu, značka zdroja, strojová kontrola, a UNVERIFIED tam, kde sa nemeralo.

11. Stav voči zadaniu a obhajoba

11.1 Nezávislý posudok: kto, ako a čo našiel

V repozitári je súbor fable-review.md s dátumom 2026-09-19. Je to **posudok funkčnosti spracovaný nezávislým agentom** (fable) nad pracovným stromom nad commitom da8ad2e. Metóda je v ňom napísaná a je podstatná: posudzovateľ prešiel vety zadania jednu po druhej, ku každej priradil, čo v repozitári zodpovedá, a **väčšinu overil spustením príkazov v repozitári** — v tabuľke je pri každom riadku stĺpec „Overil som spustením?“ s konkrétnym príkazom a jeho výstupom. Čo neoveril, je tam napísané tiež (napríklad zastavenie pri nesúlade profilu s bootom neoveroval, lebo profil v ten deň k host'ovi sedel). Posudzoval stav, ktorý beží, nie HEAD.

Výsledok z 19 položiek:

stav	počet	príklady
FUNGUJE	10	pripojenie k hypervízoru, periodický zber, parsovanie štruktúr, procesy, moduly, sokety, štatistiky pamäťových oblastí, per-bin vektory, API rozhranie
ČIASTOČNE	7	real-time introspekcia, detekcia podozrivých vzorcov, sekvenčné vektory s normalizáciou, TCN model, optimalizácia a minimalizácia vplyvu, „komplexne testovaný“ (slovo zo zadania), výkonnostné náročnosti
CHÝBA	2	vyhodnotenie presnosti detekcie malvérových vzorcov, použiteľnosť v reálnych scenároch

Záver posudku doslova: zberná a rekonštrukčná časť zadania je pokrytá a dá sa spustiť; detekčná časť pokrytá nie je.

Posudok našiel aj konkrétne trhliny, ktoré sú v tomto dokumente spracované na svojich miestach: padajúci features crosscheck bez prepínačov (časť 7.5, medzitým opravené), `root_run.sh validate` nespúšťa porovnanie (časť 7.5), živá latencia sa dá zmerať hneď (časť 8.7), dve sedenia sa nedajú reprodukovat' (časť 9.3), a `tcn/score.py` aj prepísaný profil v čase posudku neboli v gite.

11.2 Otázky na obhajobe a odpovede

„Je to naozaj hypervisor modul?“ Zadanie žiada modul postavený na open-source hypervízore a formu nepredpisuje. Na tomto stroji je KVM zavedený ako jadrový modul. Riešenie je delené rovnako: v používateľskom priestore beží `pick_vm` a `resolve_offsets` (`vmicollect/src/backend_ebpf.c`), v jadre bežia programy `vmic_probe` a `vmic_read` (`vmicollect/bpf/vmic_kvm.bpf.c`). To, že bežia v jadre, samo osebe nedokazuje nič — v jadre beží každý program eBPF. Nosné je, čo čítajú: `struct kvm` z deskriptora `kvm-vm` a jej tabuľku memslotov, teda mapu medzi fyzickou adresou host'a a virtuálnou adresou v QEMU. To je dátová štruktúra hypervízora, nie výpis z host'a. Rozbor je v `docs/ARCHITEKTURA.md`, časť 1.2.

„Prečo eBPF, keď to zadanie nespomína?“ Zadanie menuje hypervízor, nie rozhranie k nemu. Ostatné cesty k mape memslotov si pýtajú viac: LibVMI nad KVM chce patchnuté QEMU alebo jadro s KVMi, `virsh dump` vypíše vždy celý obraz naraz a zastaví stroj, a pýtať sa samotného QEMU cez QMP znamená veriť tomu, čo si VMM myslí o sebe. eBPF chce root na tom istom stroji a jadro s BTF. Druhý dôvod je prevádzkový: chyba v module .ko zhodí hostiteľa aj so všetkými virtuálnymi strojmi, kým program eBPF musí najprv prejsť verifikátorom jadra a nespustiť sa, ak sa nedá dokázať, že skončí a nepristúpi mimo pamäte.

„V čom sa to líši od `process_vm_readv`?“ V kopírovaní bajtov v ničom, a je poctivé to povedať hneď. Kopíruje sa cez `bpf_copy_from_user_task()`, čo vedie na `access_process_vm()` — tú istú funkciu jadra, ktorú používa `process_vm_readv()`. Rozdiel je v tom, čo sa vie predtým. `process_vm_readv` prečíta adresný priestor procesu QEMU, ale zvonka sa nedá zistiť, ktorá časť toho priestoru je ktorá fyzická adresa host'a; musel by sa hádať z `/proc/<pid>/maps`. Prínosom eBPF je výhradne tá mapa z memslotov.

„Čo teda váš modul deteguje?“ Zatiaľ nič, čo by som vydával za detekciu malvéru. Model má náhodné váhy a program to hlási na troch miestach vrátane každého výpisu skóre. Čo funguje, sú tri invariantné kontroly nad rekonštruovanou pamäťou: či niektorá z 451 položiek `sys_call_table` ukazuje mimo textu jadra, či sa zoznam procesov `tasks` zhoduje so stromom potomkov, a či sa zoznam `modules` zhoduje s `module_kset`. Na všetkých meraných snímkach dali 0 nálezov. To znamená, že tieto tri konkrétne invarianty neboli porušené — nie že na host'ovi nič nie je. Absencia nálezu nie je dôkaz neprítomnosti a slepé miesta týchto kontrol menuje L10. Že nula nie je ticho, drží návratový kód: 0 znamená „kontroly sa uzavreli a nič nenašli“, 4 znamená „nenašli, ale aspoň jedna sa neuzavrela“, 1 znamená nález. Detekcia naučená z dát chýba a dôvod je vstup, nie kód.

„Spustili ste aspoň jeden rootkit?“ Nie. Do host'a sa vlastný modul jadra ani načítať nedá — nemá gcc, make, hlavičky jadra ani funkčné DNS. Namiesto toho existuje pozitívny prípad, ktorý je presne opísaný ako to, čím je: v kópii reálnej snímky sa prepísalo 8 bajtov, položka 59 (`__NR_execve`) na adresu `0xfffffffffc0001000` v

oblasti modulov. Kontrola nad nedotknutou kópiou dá 0 nálezov, nad prepísanou presne 1 nález s `index=59` a `in_module_area=true`. Je to overenie, že kontrola vidí to, na čo je, nie dôkaz, že odhalí skutočný rootkit (L6). Slepé miesta sú vymenované v L10 a v `docs/kontroly.md`.

„**Ako ste zmerali vplyv na VM?**“ Nijako, a je to zapísané ako UNVERIFIED v L12. Zmerané je len to, že sa virtuálny stroj nezastavuje: `pause_ms` je 0,0 a `paused` je `false` vo všetkých 79 uložených sidecaroch. Z toho nevyplýva, že vplyv je nulový — spotrebu CPU a pamäte vnútri host'a počas zberu nikto nemeral. Preto v práci nie je veta „minimálny vplyv na virtuálny stroj“; namiesto nej stojí perióda zberu a namerané trvanie fáz cyklu. Chýbajúci benchmark je pomenovaný aj s metodikou, ktorá by ho uzavrela: pevný objem práce v host'ovi, schéma A/B/A, medián a IQR z aspoň 12 opakovaní.

„**Je to real-time?**“ Nie v zmysle garantovaného času odozvy. Je to periodický zber s meraným časom cyklu: pri perióde 2 s je p95 delta cyklu 612,6 ms ($n = 12$), pri 5 s 617,9 ms ($n = 24$), plná snímka 1588 ms. Falzifikovateľné kritérium je „p95 latencie cyklu < perióda zberu“ a to dnes platí. Zároveň platí tvrdá horná hranica časového rozlíšenia (L15): proces, ktorý vznikne a zanikne medzi dvoma snímkami, sa v rekonštruovanom zozname neobjaví — v meranom sedení vzniklo a zaniklo 8320 procesov a príznaky `proc_new` a `proc_gone` zostali na nule. Kratšia perióda hranicu posunie, neodstráni ju.

„**Čo z toho ste spravili vy?**“ Zberač `vmicollect` som nepísal riadok po riadku sám. Vznikol 26. augusta 2026 v mojom sedení s jazykovým modelom, ktoré som začal zadaním „ten periodický zber potrebujem cez eBPF a len pre KVM“; sedenie zapísalo `backend_ebpf.c`, `vmic_kvm.bpf.c`, `vmic_bpf_abi.h`, `Makefile` a `README`, ďalších päť súborov jeho podagenti. Časové značky súborov to potvrdzujú — `backend_ebpf.c` 13:27:33, `vmic_bpf_abi.h` 13:26:56, a sedenie skončilo 13:31. Formulácia „autorstvo neznáme“, ktorá je ešte v `docs/ARCHITEKTURA.md` 1.2, v `README.md` a v `HONESTY.md` P9, je nepresná a opravuje sa.

Moja práca je: zadanie a návrh toho, čo má vzniknúť a prečo práve cez eBPF nad KVM; overenie proti bežiacemu jadru, pri ktorom sa program **nenačítal**; nájdienie a oprava štyroch príčin (nezhoda BTF pri `bpf_task_from_vpid`, kde dopredná deklarácia vyrobí `FWD 'task_struct'` proti `STRUCT` vo `vmlinux`, a trikrát limit verifikátora — 8192 skokov a dvakrát 10^6 inštrukcií, všetky riešené prepisom na `bpf_loop()`); celá rekonštrukcia objektov host'a v `guestparse/` vrátane prekladu `adries`, profilu a validácie proti pozemnej pravde; príznakový vektor v C aj jeho nezávislá Python referencia; model a `baseliney` v `tcn/`; a dokumentácia vrátane priznaných obmedzení. Bez tých štyroch opráv by zberač nezbieral nič.

„**Prečo je profil jadra vstupná závislosť? Nie je to porušenie neinvazívnosti?**“ Profil sú dva vstupy z host'a: `/proc/kallsyms` (adresy symbolov) a `/sys/kernel/btf/vmlinux` (offsety polí v štruktúrach); v adresári sú uložené ako päť súborov vrátane `boot.json`, ktorý dokladá, z ktorého bootu sú. Bez nich je snímka len 2 GiB bajtov. Odoberá sa **raz a mimo meraného okna zberu**, presne ako to robí LibVMI aj Volatility. Neinvazívnosť v zmysle zadania znamená, že v host'ovi počas merania nebeží žiadny bezpečnostný agent — a to platí; agent zo staršieho projektu sa pred referenčnými meraniami ukončil 2026-09-18 o 15:29:20Z a je to doložené. Neznamená to „bez akejkoľvek znalosti host'a“; takú vetu práca nikde neobsahuje a L3 to hovorí výslovne. Výpis BTF sa robí na hostiteľovi, aby sa kvôli profilu do host'a nič neinštalovalo.

„**Čo keď sa jadro host'a zmení?**“ Profil má dve časti a každá starne inak. Offsety z BTF sú viazané na verziu jadra vrátane jeho konfigurácie a reštart prežijú. Adresy z `kallsyms` nie — jadro ich pri každom štarte posúva (KASLR), takže profil platí pre **jeden boot**. Overené: doména sa 18. septembra vypla a 19. naštartovala, `init_task` sa presunul z `0xffffffffff97a1aa40` na `0xffffffffffb221aa40`. So starým profilom sa posun jadra síce našiel, ale prechod tabuľkami stránok skončil na chyba_polozka a výpisy boli označené NEUPLNE. Nástroj teda neklamal. Od 19. septembra to nejde ani prehliadnuť: `GuestView.profile_boot_mismatch()` sa volá pred každým podpríkazom, pri nesúlade vypíše NESULAD PROFILU s pomenovanou príčinou a skončí kódom 5; `validate` sa vtedy nespustí vôbec. Riešenie je odobrať profil nanovo (`scripts/get_profile.sh -f`). Rovnaká závislosť platí pre LibVMI aj Volatility.

„**Prečo veríte tomu, čo vám parser vypíše?**“ Lebo sa to porovnáva proti tomu, čo host' hlási sám o sebe, a lebo má dve nezávislé kontroly vnútri. Krížová kontrola prekladu `adries` porovnáva lineárny výpočet s prechodom tabuliek stránok — dve nesúvisiace cesty k tej istej fyzickej adrese. Validácia proti pozemnej pravde porovnáva rekonštrukciu s výstupom `ps`, `lsmod` a `ss` z host'a, pričom sa počíta iba stabilná množina a každé pravidlo zhody mien sa vykazuje zvlášť. A kotva posunu jadra sa overuje druhým, nesúvisiacim symbolom (`swapper/0`), nie sama sebou.

12. Slovníček

Pojmy sú v texte vysvetlené pri prvom použití; táto tabuľka je na rýchle dohľadanie.

Virtualizácia a hypervízor

pojmem	čo to je
hypervízor	softvér, ktorý rozdeľuje jeden fyzický počítač na viac virtuálnych a dohliada na ne; má privilegovanejší pohľad než operačný systém vnútri nich
KVM	modul jadra Linuxu, ktorý z jadra robí hypervízor; tu zavedený ako <code>kvm</code> a <code>kvm_intel</code>
QEMU	proces v používateľskom priestore, ktorý virtuálnemu stroju emuluje disk, sieť a obrazovku; pamäť host'a je alokovaná v jeho adresnom priestore
VMM	„virtual machine monitor“, spoločné meno pre proces ako QEMU, ktorý konkrétny virtuálny stroj vedie
host' / hostiteľ	host' je operačný systém vo virtuálnom stroji (tu Debian 12), hostiteľ je fyzický stroj pod ním (tu Fedora 43)
memslot	jeden záznam v tabuľke KVM: „tento rozsah fyzickej pamäte host'a leží na tejto adrese v QEMU“; doména <code>hypocn-guest</code> ich má 10
anonymný inode	záznam o „súbore“, ktorý na disku nie je a nemá meno v adresári; proces ho vidí ako <code>anon_inode: <meno></code>
mmap	KVM ním vyrába rukoväť na VM (<code>kvm-vm</code>) a na vCPU systémové volanie, ktorým proces požiada jadro o vloženie oblasti pamäte do svojho adresného priestoru; anonymné mapovanie je čistá prázdna pamäť bez súboru
MMIO	memory-mapped I/O: riadiace registre zariadení sprístupnené ako pamäťové adresy
PCI hole	rozsah fyzických adries pod 4 GiB rezervovaný pre MMIO zariadení; nie je tam RAM

Adresy a pamäť

pojmem	čo to je
GPA	fyzická adresa host'a — adresa, ako ju vidí jadro host'a vo svojej RAM
HVA	virtuálna adresa hostiteľa — kde tá istá pamäť leží v adresnom priestore procesu QEMU
GVA	virtuálna adresa host'a — adresa, ako ju vidí program alebo jadro vnútri host'a
stránka	najmenší kus pamäte, s ktorým procesor pracuje pri mapovaní; tu 4096 bajtov, VM ich má 528 417
gfn	guest frame number: poradové číslo stránky vo fyzickej pamäti host'a, teda GPA delená 4096. Pole <code>base_gfn</code> v memslot je začiatok bloku vyjadrený v stránkach
tabuľky stránok	štvorúrovňová stromová štruktúra (<code>PGD → PUD → PMD → PTE</code>), ktorou jadro prekladá virtuálnu adresu na fyzickú; prechod ňou sa dá zo snímky zopakovať
__START_KERNEL_map	konštanta <code>0xffffffff80000000</code> , virtuálna adresa, na ktorú je namapovaný obraz jadra. Jej odčítaním od virtuálnej adresy dostaneme posun vnútri obrazu jadra, ktorý sa potom prepočíta na fyzickú adresu
priamy mapping	okno vo virtuálnom priestore jadra, do ktorého je lineárne namapovaná celá fyzická RAM stroja; jeho začiatok drží premenná <code>page_offset_base</code> , takže prevod je jedno odčítanie

pojem	čo to je
veľká stránka	mapovanie 2 MiB (na úrovni PMD) alebo 1 GiB (PUD) namiesto 4 KiB; pozná sa podľa bitu PSE
KASLR	náhodné posunutie jadra v pamäti pri každom štarte; dôvod, prečo profil platí len pre jeden boot
riedky (sparse) súbor	súbor, ktorého nezapísané bloky na disku neexistujú a pri čítaní sa vracajú ako nuly; hash nad ním musí tie nuly vyrobiť, hoci sa nikdy nezapísali

eBPF a zberač

pojem	čo to je
eBPF	malé programy nahrané do jadra, ktoré tam bežia v piesočku; používajú sa na sledovanie a meranie bez prekladu vlastného jadra
verifikátor	časť jadra, ktorá program eBPF pred spustením overí; keď nedokáže dokázať, že skončí a nepristúpi mimo pamäte, program odmietne
BPF mapa	dátová štruktúra vlastnená jadrom, do ktorej program v jadre zapisuje a číta, a ktorú súčasne vidí aj používateľský proces cez deskriptor súboru; tu <code>ko</code> ff (offsety dnu) a <code>pages</code> (stránky von)
BPF_F_MMAPABLE	príznak mapy, ktorý dovolí používateľskému procesu namapovať ju priamo do svojho adresného priestoru cez <code>mmap</code>
bpf_prog_test_run_opts()	rozhranie <code>libbpf</code> , ktorým si používateľský proces spustí načítaný BPF program vtedy, keď chce, bez toho, aby ho na niečo vešal
__ksym	značka v zdrojáku BPF programu: „tento symbol je v jadre, nie v mojom objekte“; <code>libbpf</code> ho pri načítaní vyhľadá v BTF jadra a porovná typy
bpf_loop()	pomocná funkcia, ktorá dá telo cyklu verifikátorovi overiť raz namiesto tisíckrát; ňou sa tu riešili tri zo štyroch nájdených chýb
BTF	typové informácie zabudované v jadre; z nich sa čítajú offsety polí v štruktúrach, aby neboli zadrátované v kóde
RCU	Read-Copy-Update: spôsob čítania zdieľaných štruktúr jadra bez zámku. Čitateľ sa ohlásí vstupom do čítacej sekcie a jadro dovedy neuvoľní staré uzly; preto sa tabuľky čítajú pod <code>bpf_rcu_read_lock()</code>
hašovacia tabuľka, bucket	tabuľka, v ktorej sa záznam nájde prepočítaním kľúča na číslo priečinka; priečinok je bucket a visí v ňom krátky reťazec záznamov s rovnakým výsledkom. <code>id_hash</code> memslotov má 128 bucketov
generation	počítadlo, ktoré KVM zvýši pri každej zmene tabuľky memslotov; keď sa počas prechodu zmení, cyklus sa zahodí
hook	zásuvný modul zberača načítaný cez <code>dlopen</code> ; tu notifikačný — dostane cestu k hotovej snímke a časy, nie obsah pamäte
ABI	dohoda o tom, ako presne vyzerajú funkcie a štruktúry v preloženom kóde; hook API nesie číslo <code>abi</code> , aby sa nesprávala dvojica zberač–plugin z rôznych verzií hlavičky

pojmem	čo to je
backend file	druhý backend, ktorý číta obraz pamäte zo súboru; nepotrebuje root ani VM, stoja na ňom testy a ukážka hooku
retencia	automatické mazanie starých snímok podľa počtu, veľkosti alebo veku; maže po celých reťazcoch, lebo delta bez svojej plnej snímky sa nedá obnoviť

Snímky a formát

pojmem	čo to je
snímka (snapshot)	kópia obsahu pamäte VM v jednom okamihu; tu vo vlastnom formáte .vmicd
plná snímka	snímka, ktorá obsahuje každú nenulovú stránku pamäte; je referenciou pre delty za ňou
delta snímka	snímka, ktorá ukladá iba stránky zmenené od predchádzajúcej; preto rádovo 1 MiB namiesto stoviek MiB
reťazec (chain)	plná snímka a všetky delty, ktoré sa o ňu opierajú; má chain_id a snímky v ňom seq
baseline snímka	prvá snímka reťazca, teda tá plná; bez nej sa zvyšok reťazca obnoviť nedá
sidecar	JSON súbor vedľa snímky s metadátami: časy, počty stránok, kontrolné súčty, zbierané oblasti a príznakový vektor; schéma vmicollect/1
SHA-NI	inštrukcie procesora určené priamo pre SHA-256; tu 2150,7 MiB/s proti 357,3 MiB/s bez nich

Semantic gap a parser

pojmem	čo to je
semantic gap	priepasť medzi „mám 2 GiB bajtov“ a „viem, že toto je proces sshd s pid 381“; preklenúť ju je celá práca parsera
kallsyms	zoznam „meno symbolu → adresa“ z bežiacého jadra host'a (/proc/kallsyms)
profil jadra	kallsyms + BTF pre konkrétny boot host'a; v adresári je päť súborov vrátane boot.json, ktorý dokladá, z ktorého štartu profil je
task_struct	štruktúra jadra Linuxu, ktorá opisuje jeden proces; zoznam procesov je obojsmerne viazaný reťazec týchto štruktúr
container_of	štandardný idiom jadra: od ukazovateľa na člen štruktúry k ukazovateľu na celú štruktúru, jedným odčítaním offsetu. Tu odčítanie čísla 2192 (offset poľa tasks)
pozemná pravda	to, čo host' o sebe hlási sám (ps, lsmode, ss) v čase snímky; s tým sa porovnáva rekonštrukcia
ss -tuap vs. ss -tulpn	prvý vypisuje všetky sokety TCP a UDP vrátane nadviazaných spojení, druhý iba počívajúce ; miešať ich v jednej tabuľke sa nesmie
stabilná množina	procesy, ktoré sú v ps pred snímkom aj po nej; iba tie sa porovnávajú
NEUPLNE	označenie výpisu, ktorého prechod zoznamom sa neuzavrel; výsledok je dolná hranica

Príznamy a model

pojmem	čo to je
bin	pevne veľký kus fyzickej pamäte (tu 16 MiB), za ktorý sa počítajú štatistiky; doména má 131 binov, lebo sa binuje len nad memslotmi
Shannonova entropia	číslo 0–8 hovoriace, koľko bitov na bajt nesie obsah stránky; zašifrované a komprimované dáta majú vysokú, nuly nízku
mem_changed_spread	druhá veličina s menom entropia: normalizovaná entropia rozdelenia zmien medzi biny, 0 až 1. Nie je to entropia obsahu, ale rovnomernosť rozloženia zmien po pamäti
príznam (feature)	jedno číslo opisujúce snímku, napríklad <code>proc_total</code> alebo <code>mem_entropy_mean</code> ; z jednej snímky vzniká vektor 22 čísel
okno	sekvencia po sebe idúcich snímok, ktorú dostane model naraz; východzia dĺžka je 16
sedenie (session)	jeden súvislý beh zberu nad jednou doménou; najmenšia jednotka, ktorá sa pri delení dát nesmie rozrezať
split	rozdelenie dát na tréning a test; tu chronologický a session-disjunktný
fit / transform	<code>fit</code> = nauč sa parametre z dát (pri z-score priemer a odchýlku), <code>transform</code> = použi ich na dáta. <code>transform</code> pred <code>fit</code> vyhodí výnimku
manifest	súbor s uloženým fitom: priemery, odchýlky, poradie príznamov a jeho hash, aby sa na testovacích dátach použili tie isté hodnoty
z-score	normalizácia (hodnota – priemer) / smerodajná odchýlka, aby príznamy v rôznych jednotkách mali porovnateľnú škálu
data leakage	keď sa do tréningu dostane informácia z testu (napríklad okná tej istej relácie v oboch, alebo μ/σ počítané zo všetkých dát), takže výsledok vyzerá lepší, než je
confound	sprievodná vlastnosť, ktorá koreluje s triedou a model sa naučí ju namiesto javu; spoločný dôvod neplatnosti čísel z <code>hypTcn002</code>
konvolúcia	operácia, ktorá po sekvencii posúva malé okienko váh a počíta z neho výstup; v čase hľadá lokálne vzory
kanály	počet nezávislých filtrov v konvolúcii, a teda počet výstupných postupností; tu 16
kauzálna konvolúcia	konvolúcia, ktorá sa pozerá len dozadu — výstup pre čas t nikdy nepoužije snímku z času $t+1$
dilatácia	preskakovanie vstupov v konvolúcii (1, 2, 4, ...), aby sa pri malom počte vrstiev videlo ďaleko do minulosti
reziduálny blok	blok, ktorého výstup sa pripočíta k vstupu ($f(x) + x$); vrstva sa učí len rozdiel, model sa vie naučiť „nič nerobiť“ a gradient má skratku
dropout	náhodné vynulovanie časti hodnôt počas tréningu (tu 10 %) ako zábrana proti preučeniu; pri vyhodnotení sa vypína
gradient	citlivosť výstupu na malú zmenu každej váhy; používa sa pri učení a tu aj na meranie receptívneho poľa
receptive field (RF)	koľko časových krokov dovidí posledný výstup modelu; $RF = 1 + 2(k-1)(2^B - 1) = 29$ pri jadre 3 a troch blokoch, teda viac než okno 16
logity	surové čísla na výstupe poslednej vrstvy, jedno na triedu; môžu byť záporné a nedávajú súčet 1

pojem	čo to je
softmax	funkcia, ktorá logity prevedie na pravdepodobnosti: umocní ich na e a vydolí súčtom, takže výsledok je kladný a sčíta sa na 1
TCN	Temporal Convolutional Network — sieť z reziduálnych blokov kauzálnych dilatovaných konvolúcií; alternatíva k LSTM pre sekvencie
baseline (model)	jednoduchší model, proti ktorému sa TCN meria; tu logistická regresia, bag-of-frames a GRU
bag-of-frames	baseline, ktorý okno spriemeruje a zahodí poradie snímok; keď TCN nie je lepšia než on, čas v dátach nič nenesie

Meranie a poctivosť

pojem	čo to je
medián	prostredná hodnota zoradeného radu; jeden výkyv ňou nepohne, na rozdiel od priemeru
p95	hodnota, pod ktorou leží 95 % meraní; hovorí, aký zlý je zlý prípad
IQR	medzikvartilové rozpätie, rozdiel 75. a 25. percentilu — šírka pásma, v ktorom leží prostredná polovica meraní
artefakt	uložený súbor v <code>data/results/</code> , z ktorého sa číslo dá znova prečítať; číslo bez neho sa v práci neuvádza
značka zdroja	zápis <code>{{res:subor.json:kluc}}</code> hneď za číslom; <code>check_claims.sh</code> overí súbor, kľúč aj hodnotu
UNVERIFIED	značka pre veličinu, ktorá sa v tejto práci nemerala a nesmie sa uviesť ako výsledok, kým nevznikne artefakt
RETRAKCIA	slovo na riadku, ktoré dovolí uviesť zakázané alebo neplatné číslo spolu s vysvetlením, čo ho zneplatnilo
značka (A) / (Z)	v tabuľkách kapitoly 8: (A) = meranie vzniklo, keď v hosťovi ešte bežal agent zo staršieho projektu; (Z) = zmrazený hosť bez agenta, teda po 2026-09-18T15:29:20Z